

# Revisit to CPE233 (Digital Design)

**Authors:**  
WhisperingRock

**Professor:**

Class  
Quarter  
July 28, 2026

# Contents

|       |                              |    |
|-------|------------------------------|----|
| 0.1   | Hardware                     | 2  |
| 0.1.1 | HW1 - Program ROM            | 2  |
| 0.1.2 | HW2 - Program Counter        | 5  |
| 0.1.3 | HW3 - Register File          | 11 |
| 0.1.4 | HW4 - ALU + ImmGen           | 15 |
| 0.2   | Software                     | 25 |
| 0.2.1 | SW1 - Intro to Assembly Lang | 25 |
| 0.2.2 | SW2 - Conditional Statements | 34 |
| 0.2.3 | SW3 - Loops                  | 38 |
| 0.2.4 | SW4 - Arrays                 | 43 |
| 0.2.5 | SW5 - Division and Stacks    | 49 |
| 0.3   | Appendix                     | 55 |

Note : All assignments are in the CPE 233 Lab Manual.

0.1 Hardware

0.1.1 HW1 - Program ROM

Reverse Engineer’d Table

|              |          |
|--------------|----------|
| 0x0000_0000: | 11000537 |
| 0x0000_0004: | 00a00f13 |
| 0x0000_0008: | 00051783 |
| 0x0000_000C: | 41e7da33 |
| 0x0000_0010: | 00fa4633 |
| 0x0000_0014: | 04c52023 |
| 0x0000_0018: | ff1ff06f |

Figure 1: Which Table? This table

RISC-V Instruction Set Summary

|                     |        |        |        |           |        |
|---------------------|--------|--------|--------|-----------|--------|
| 31:25               | 24:20  | 19:15  | 14:12  | 11:7      | 6:0    |
| funct7              | rs2    | rs1    | funct3 | rd        | op     |
| imm11,0             |        | rs1    | funct3 | rd        | op     |
| imm11,5             | rs2    | rs1    | funct3 | imm4,0    | op     |
| imm12,10,5          | rs2    | rs1    | funct3 | imm4,1,11 | op     |
| imm31,12            |        |        |        | rd        | op     |
| imm20,10,1,11,19,12 |        |        |        | rd        | op     |
| fs3                 | funct2 | fs2    | fs1    | funct3    | fd     |
| 5 bits              | 2 bits | 5 bits | 3 bits | 5 bits    | 7 bits |

**R-Type**

**I-Type**

**S-Type**

**B-Type**

**U-Type**

**J-Type**

**R4-Type**

- imm: signed immediate in imm<sub>11,0</sub>
- uimm: 5-bit unsigned immediate in imm<sub>4,0</sub>
- upimm: 20 upper bits of a 32-bit immediate, in imm<sub>31,12</sub>
- Address: memory address: rs1 + SignExt(imm<sub>11,0</sub>)
- [Address]: data at memory location Address
- BTA: branch target address: PC + SignExt((imm<sub>12,1,1'b0</sub>))
- JTA: jump target address: PC + SignExt((imm<sub>20,1,1'b0</sub>))
- label: text indicating instruction address
- SignExt: value sign-extended to 32 bits
- ZeroExt: value zero-extended to 32 bits
- csr: control and status register

**Figure B.1 RISC-V 32-bit instruction formats**

Table B.1 RV32I: RISC-V integer instructions

| op            | funct3 | funct7   | Type | Instruction          | Description                   | Operation                            |
|---------------|--------|----------|------|----------------------|-------------------------------|--------------------------------------|
| 0000011 (3)   | 000    | –        | I    | lb rd, imm(rs1)      | load byte                     | rd = SignExt([Address]7:0)           |
| 0000011 (3)   | 001    | –        | I    | lh rd, imm(rs1)      | load half                     | rd = SignExt([Address]15:0)          |
| 0000011 (3)   | 010    | –        | I    | lw rd, imm(rs1)      | load word                     | rd = [Address]31:0                   |
| 0000011 (3)   | 100    | –        | I    | lbu rd, imm(rs1)     | load byte unsigned            | rd = ZeroExt([Address]7:0)           |
| 0000011 (3)   | 101    | –        | I    | lhu rd, imm(rs1)     | load half unsigned            | rd = ZeroExt([Address]15:0)          |
| 0010011 (19)  | 000    | –        | I    | addi rd, rs1, imm    | add immediate                 | rd = rs1 + SignExt(imm)              |
| 0010011 (19)  | 001    | 0000000* | I    | slli rd, rs1, uimm   | shift left logical immediate  | rd = rs1 << uimm                     |
| 0010011 (19)  | 010    | –        | I    | slti rd, rs1, imm    | set less than immediate       | rd = (rs1 < SignExt(imm))            |
| 0010011 (19)  | 011    | –        | I    | sltiu rd, rs1, imm   | set less than imm. unsigned   | rd = (rs1 < SignExt(imm))            |
| 0010011 (19)  | 100    | –        | I    | xori rd, rs1, imm    | xor immediate                 | rd = rs1 ^ SignExt(imm)              |
| 0010011 (19)  | 101    | 0000000* | I    | srlr rd, rs1, uimm   | shift right logical immediate | rd = rs1 >> uimm                     |
| 0010011 (19)  | 101    | 0100000* | I    | srair rd, rs1, uimm  | shift right arithmetic imm.   | rd = rs1 >>> uimm                    |
| 0010011 (19)  | 110    | –        | I    | orir rd, rs1, imm    | or immediate                  | rd = rs1   SignExt(imm)              |
| 0010011 (19)  | 111    | –        | I    | andir rd, rs1, imm   | and immediate                 | rd = rs1 & SignExt(imm)              |
| 0010111 (23)  | –      | –        | U    | auipc rd, upimm      | add upper immediate to PC     | rd = (upimm, 12'b0) + PC             |
| 0100011 (35)  | 000    | –        | S    | sb rs2, imm(rs1)     | store byte                    | [Address]7:0 = rs27:0                |
| 0100011 (35)  | 001    | –        | S    | sh rs2, imm(rs1)     | store half                    | [Address]15:0 = rs215:0              |
| 0100011 (35)  | 010    | –        | S    | sw rs2, imm(rs1)     | store word                    | [Address]31:0 = rs2                  |
| 0110011 (51)  | 000    | 0000000  | R    | add rd, rs1, rs2     | add                           | rd = rs1 + rs2                       |
| 0110011 (51)  | 000    | 0100000  | R    | sub rd, rs1, rs2     | sub                           | rd = rs1 – rs2                       |
| 0110011 (51)  | 001    | 0000000  | R    | sll rd, rs1, rs2     | shift left logical            | rd = rs1 << rs24:0                   |
| 0110011 (51)  | 010    | 0000000  | R    | slt rd, rs1, rs2     | set less than                 | rd = (rs1 < rs2)                     |
| 0110011 (51)  | 011    | 0000000  | R    | sltu rd, rs1, rs2    | set less than unsigned        | rd = (rs1 < rs2)                     |
| 0110011 (51)  | 100    | 0000000  | R    | xor rd, rs1, rs2     | xor                           | rd = rs1 ^ rs2                       |
| 0110011 (51)  | 101    | 0000000  | R    | srl rd, rs1, rs2     | shift right logical           | rd = rs1 >> rs24:0                   |
| 0110011 (51)  | 101    | 0100000  | R    | sra rd, rs1, rs2     | shift right arithmetic        | rd = rs1 >>> rs24:0                  |
| 0110011 (51)  | 110    | 0000000  | R    | or rd, rs1, rs2      | or                            | rd = rs1   rs2                       |
| 0110011 (51)  | 111    | 0000000  | R    | and rd, rs1, rs2     | and                           | rd = rs1 & rs2                       |
| 0110111 (55)  | –      | –        | U    | lui rd, upimm        | load upper immediate          | rd = (upimm, 12'b0)                  |
| 1100011 (99)  | 000    | –        | B    | beq rs1, rs2, label  | branch if =                   | if (rs1 == rs2) PC = BTA             |
| 1100011 (99)  | 001    | –        | B    | bne rs1, rs2, label  | branch if ≠                   | if (rs1 ≠ rs2) PC = BTA              |
| 1100011 (99)  | 100    | –        | B    | blt rs1, rs2, label  | branch if <                   | if (rs1 < rs2) PC = BTA              |
| 1100011 (99)  | 101    | –        | B    | bge rs1, rs2, label  | branch if ≥                   | if (rs1 ≥ rs2) PC = BTA              |
| 1100011 (99)  | 110    | –        | B    | bltu rs1, rs2, label | branch if < unsigned          | if (rs1 < rs2) PC = BTA              |
| 1100011 (99)  | 111    | –        | B    | bgeu rs1, rs2, label | branch if ≥ unsigned          | if (rs1 ≥ rs2) PC = BTA              |
| 1100111 (103) | 000    | –        | I    | jalr rd, rs1, imm    | jump and link register        | PC = rs1 + SignExt(imm), rd = PC + 4 |
| 1101111 (111) | –      | –        | J    | jal rd, label        | jump and link                 | PC = JTA, rd = PC + 4                |

\*Encoded in instr31,25: the upper seven bits of the immediate field

| ProgROM Addr | Machine Code                               | Type   | Assembly Instruction |
|--------------|--------------------------------------------|--------|----------------------|
| 0000         | 0x1100 0537                                |        |                      |
|              | 0001 0001 0000 0000 0000 0101 0 011 0111   | (55) U | lui rd, upimm        |
|              | 0001 0001 0000 0000 0000 0101 0 011 0111   | (55) U | lui x10, 0x11000     |
| 0004         | 0x00A0 0F13                                |        |                      |
|              | 0000 0000 1010 0000 0 000 1111 0 001 0011  | (19)I  | addi rd, rs1, imm    |
|              | 0000 0000 1010 0000 0 000 1111 0 001 0011  | (19)I  | addi x30, x0, 0x00A  |
| 0008         | 0x0005 1783                                |        |                      |
|              | 0000 0000 0000 0101 0 001 0111 1 000 0011  | (3)I   | lh rd, imm(rs1)      |
|              | 0000 0000 0000 0101 0 001 0111 1 000 0011  | (3)I   | lh x15, 0(x10)       |
| 000C         | 0x41E7 DA33                                |        |                      |
|              | 0100 000 1 1110 0111 1 101 1010 0 011 0011 | (51)R  | sra rd, rs1, rs2     |
|              | 0100 000 1 1110 0111 1 101 1010 0 011 0011 | (51)R  | sra x20, x15, x30    |
| 0010         | 0x00FA 4633                                |        |                      |
|              | 0000 0000 1111 1010 0 100 0110 0 011 0011  | (51)R  | xor rd, rs1, rs2     |
|              | 0000 000 0 1111 1010 0 100 0110 0 011 0011 | (51)R  | xor x12, x20, x15    |
| 0014         | 0x04C5 2023                                |        |                      |
|              | 0000 0100 1100 0101 0 010 0000 0 010 0011  | (35)S  | sw rs2, imm(rs1)     |
|              | 0000 010 0 1100 0101 0 010 0000 0 010 0011 | (35)S  | sw x12, 0x040(x10)   |
| 0018         | 0xFF1F F06F                                |        |                      |
|              | 1111 1111 0001 1111 1111 0000 0 110 1111   | (111)J | jal rd, label        |
|              | 1111 1111 0001 1111 1111 0000 0 110 1111   | (111)J | jal x0, -16          |

Table 1: Reverse Engineering *otter\_memory.mem* segment

For the last command, some explanation is required here. Consider

- $Imm[20] = B31 = 1$
- $Imm[10 : 1] = B30 : B21 = 1111111000$
- $Imm[11] = B20 = 1$
- $Imm[19 : 12] = B19 : B12 = 11111111$
- $Imm[0]$  is always zero
- Therefore,  $Imm = 0x1FFFF0$

To find the decimal representation, consider adding our large negative number to the next highest max.

$$\begin{aligned}
 2^{21} - 0x1FFFF0 &= 0x200000 - 0x1FFFF0 \\
 &= -0x10 \\
 &= -16
 \end{aligned}$$

BUT WAIT! jal doesn't take immediates, only labels.

True, our label must be 4 spaces prior to this jump - fill this portion in with a label in the asm and respect the spacing.

### Assembly Code

Listing 1: Assembly of HW1

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Confirm decoded machine code (by hand)
4 #           functionality matches what the RARS compiler
5 #           produces.
6
7
8
9 .data
10 A: .word 0x1234    # dummy value to see change
11
12 .global _start
13
14 .text
15 .equ MMIO, 0x11000000
16
17 _start:
18 # ~~~~ init registers ~~~~
19 li t1, MMIO
20 lw t2, A
21 sw t2, 0(t1)
22
23 # ~~~~ decoded instr ~~~~

```

```
24  #lui x10, 0x11000
25  #addi x30, x0, 0x00A
26  #loop:
27  #lh x15, 0(x10)
28  #sra x20, x15, x30
29  #xor x12, x20, x15
30  #sw x12, 0x040(x10)
31  #jal x0, loop    #loop label is 4 spaces back
32
33  lui a0, 0x11000    # a0 = x10
34  addi t5, zero, 0x00A    # zero = x0, t5 = x30
35  loop:
36  lh a5, 0(a0)        # a5 = x15
37  sra s4, a5, t5        # s4 = x20
38  xor a2, s4, a5        # a2 = x12
39  sw a2, 64(a0)
40  jal zero, loop
41
42  # ~~~~ syscall exit status ~~~~
43  li a7, 10    # load syscall call num for exit()
44  li a0, 0     # EXIT_SUCCESS code
45  ecall       # make the system call
```

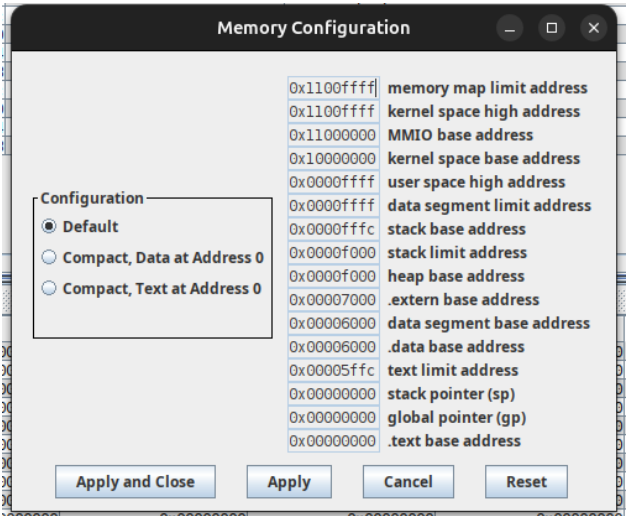


Figure 2: Segment Address Config

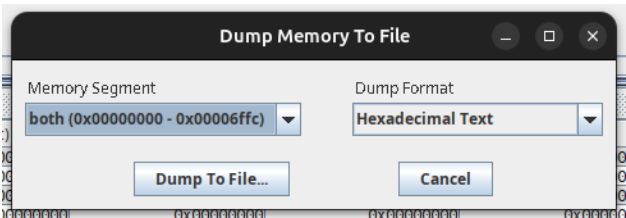


Figure 3: Memory Dump Config

| Text Segment |            |            |                         |                                                  |
|--------------|------------|------------|-------------------------|--------------------------------------------------|
| Bkpt         | Address    | Code       | Basic                   | Source                                           |
|              | 0x00000000 | 0x11000337 | lui x6, 0x00011000      | 19: li t1, 0x10000000                            |
|              | 0x00000004 | 0x00030313 | addi x6, x6, 0          |                                                  |
|              | 0x00000008 | 0x00006397 | auipc x7, 6             | 20: lw t2, A                                     |
|              | 0x0000000c | 0xff83a383 | lw x7, 0xfffffff8(x7)   |                                                  |
|              | 0x00000010 | 0x00732023 | sw x7, 0(x6)            | 21: sw t2, 0(t1)                                 |
|              | 0x00000014 | 0x11000537 | lui x10, 0x00011000     | 33: lui a0, 0x11000 # a0 = x10                   |
|              | 0x00000018 | 0x00a00f13 | addi x30, x0, 10        | 34: addi t5, zero, 0x00A # zero = x0, t5 = x30   |
|              | 0x0000001c | 0x00051783 | lh x15, 0(x10)          | 36: lh a5, 0(a0) # a5 = x15                      |
|              | 0x00000020 | 0x41e7da33 | sra x20, x15, x30       | 37: sra s4, a5, t5 # s4 = x20                    |
|              | 0x00000024 | 0x00fa4633 | xor x12, x20, x15       | 38: xor a2, s4, a5 # a2 = x12                    |
|              | 0x00000028 | 0x04c52023 | sw x12, 0x00000040(x10) | 39: sw a2, 64(a0)                                |
|              | 0x0000002c | 0xff1ff06f | jal x0, 0xfffffff0      | 40: jal zero, loop                               |
|              | 0x00000030 | 0x00a00893 | addi x17, x0, 10        | 43: li a7, 10 # load syscall call num for exit() |
|              | 0x00000034 | 0x00000513 | addi x10, x0, 0         | 44: li a0, 0 # EXIT_SUCCESS code                 |
|              | 0x00000038 | 0x00000073 | ecall                   | 45: ecall # make the system call                 |

Figure 4: RARS instruction addresses alongside code

Vivado Simulation

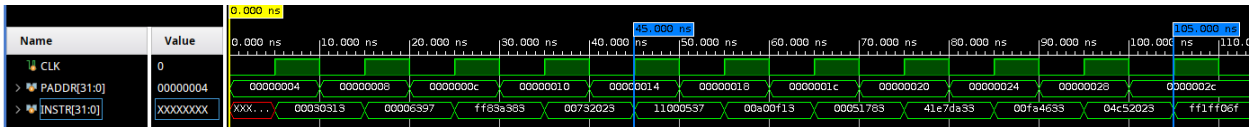


Figure 5: Simulation waveform of ProgROM\_tb.sv running otter\_memory\_hw1.mem

By inspection, the instructions in the waveform 5 match those in RARS4 and our decoded table 1

0.1.2 HW2 - Program Counter

Intended Behavior(s)

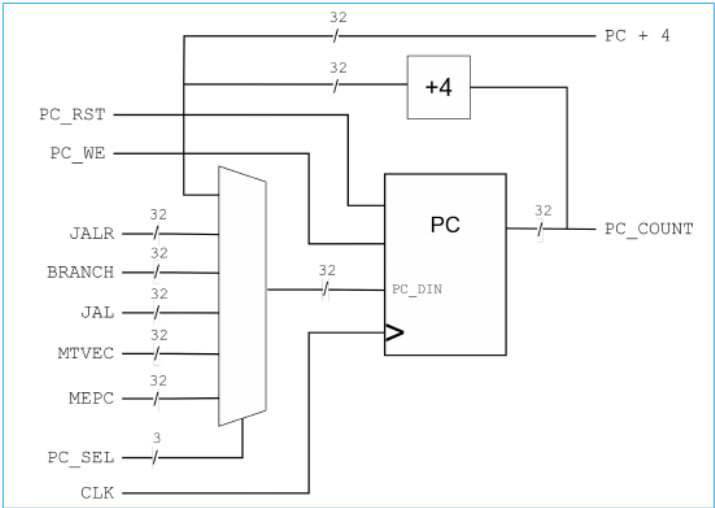


Figure 6: Program Counter Test Block Design

Structural Design(s)

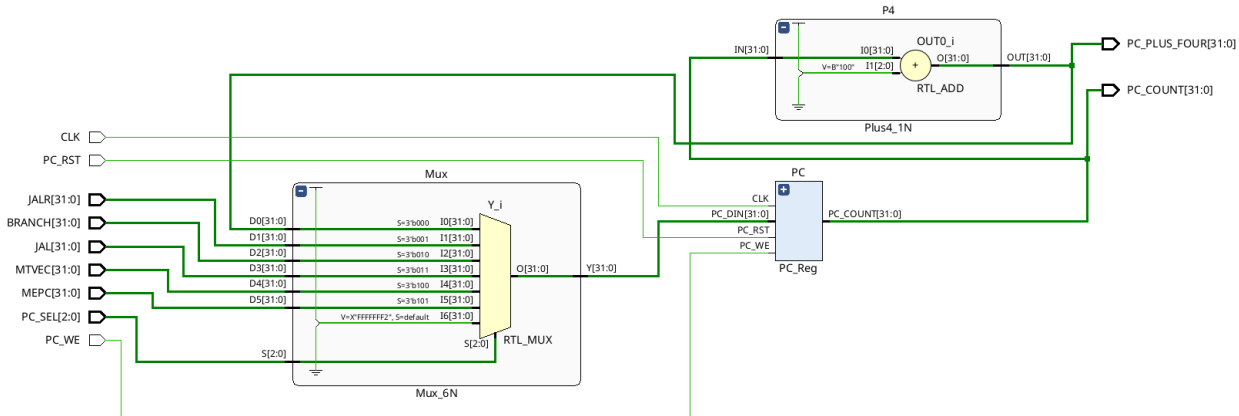


Figure 7: RTL Design of PC\_TestBlock.sv

Synthesis Warning(s)

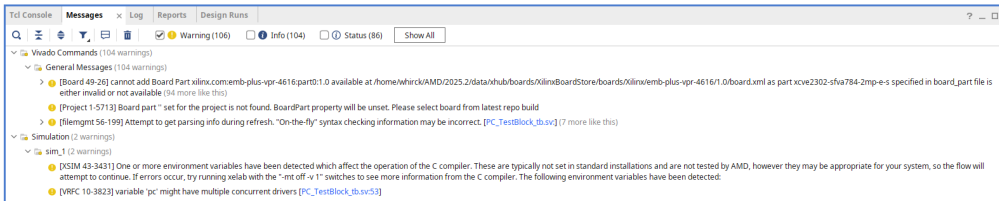


Figure 8: Synthesis Errors of PC\_TestBlock.sv

The only error with importance here is 'clk' having multiple concurrent drivers such that one clock toggles clk and the other initially defines the clk value. I opted to proceed with this error.

Verification

All test cases passed in PC\_TestBlock\_tb.sv passed in simulation along with each subcomponent test bench.

Listing 2: Simulation file for PC\_TestBlock.sv

```
1 'timescale 1ns / 1ps
2 //////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 07/19/2026 09:39:05 AM
7 // Design Name:
8 // Module Name: PC_TestBlock_tb
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
```

```

13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module PC_TestBlock_tb();
24
25     // ~~~~ init local vars ~~~~
26     logic reset, write, clk;
27     logic[2:0] sel;
28     logic[5:0][31:0] d;           // d[0] is unused b/c its p+4 on mux
29     logic[31:0] pc, pc_plus_4;
30
31     // ~~~~ instances ~~~~
32     PC_TestBlock UUT(
33         .PC_RST(reset),
34         .PC_WE(write),
35         .CLK(clk),
36         .PC_SEL(sel),
37         .JALR(d[1]),
38         .BRANCH(d[2]),
39         .JAL(d[3]),
40         .MTVEC(d[4]),
41         .MEPC(d[5]),
42         .PC_COUNT(pc),
43         .PC_PLUS_FOUR(pc_plus_4)
44     );
45
46     // ~~ action : CLK (10ns period) ~~
47     always begin
48         #5;
49         clk <= !clk;
50     end
51
52     // ~~~~ action : data ~~~~
53     initial begin
54
55         // ~~ init ~~
56         clk = 1; reset = 0; write = 0; sel = 0;
57         d[1] = 32'h1234_5678;
58         d[2] = 32'hBAAC_4444;
59         d[3] = 32'h0000_01A1;
60         d[4] = 32'hBEEF_BEEF;
61         d[5] = 32'hDEAD_DEAD;
62         pc = 0; pc_plus_4 = 0;
63         #10;
64
65         // ~~ TC1 : sel all w/ write disabled ~~
66         for (int i = 0; i < 16; i++) begin
67             sel = i;
68             #10;
69             assert(pc === 0) else $error("TC1: sel failed (pc)");
70             assert(pc_plus_4 === 4) else $error("TC1: sel failed (pc+4)");
71         end
72
73
74
75         // ~~ TC2 : sel all w/ write enabled ~~
76         reset = 1; sel = 0; #10; reset = 0; #10;
77         write = 1; #10;
78
79         assert(pc === 4) else $error("TC2: sel(0) failed on pc");           // Not zero b/c d[0] is 0+4 already
80         assert(pc_plus_4 === 8) else $error("TC2: sel(0) failed on pc+4");
81
82         for(int i=1; i < 6; i++) begin
83             sel = i; #10;
84             assert(pc === d[i]) else $error("TC2: failed on pc");
85             assert(pc_plus_4 === (d[i]+4) ) else $error("TC2: failed on pc+4");
86         end
87
88
89         // ~~ TC3 : inc pc ~~
90         reset = 1; sel = 0; #10; reset = 0;
91         for(int i=1; i < 256; i++) begin
92             #10;
93             assert(pc === (4*i)) else $error("TC3: failed on pc");
94             assert(pc_plus_4 === (4+(4*i)) ) else $error("TC3: failed on pc+4");
95         end
96
97
98         // ~ test conclusion ~
99         reset = 1;
100     end
101 endmodule

```

Listing 3: Simulation file for Mux\_6N.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/17/2026 04:26:27 PM
7  // Design Name:
8  // Module Name: Mux_6N_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module Mux_6N_tb();
24
25     // ~~ local variables ~~
26     logic CLK;
27     logic[31:0] d0, d1, d2, d3, d4, d5;
28     logic[3:0] sel;
29     logic[31:0] out;
30
31
32     // ~~ high-level module instances (and hooks) ~~
33     Mux_6N #(32) UUT (
34         .D0(d0),
35         .D1(d1),
36         .D2(d2),
37         .D3(d3),
38         .D4(d4),
39         .D5(d5),
40         .S(sel),
41         .Y(out)
42     );
43
44     // ~~ action : CLK (10ns period) ~~
45     always begin
46         #5;
47         CLK <= !CLK;
48     end
49
50     // ~~ action : Data ~~
51     initial begin
52
53         // ~ init ~
54         CLK = 0;
55         d0 = 32'h0000_0000;
56         d1 = 32'h1234_0000;
57         d2 = 32'h0000_5678;
58         d3 = 32'h1111_1111;
59         d4 = 32'hFFFF_FFFF;
60         d5 = 32'hDEAD_BEEF;
61
62         // ~ selector test ~
63         sel = 0; #10
64         assert(out === d0) else $error("sel = 0 failed");
65
66         sel = 1; #10
67         assert(out === d1) else $error("sel = 1 failed");
68
69         sel = 2; #10
70         assert(out === d2) else $error("sel = 2 failed");
71
72         sel = 3; #10
73         assert(out === d3) else $error("sel = 3 failed");
74
75         sel = 4; #10
76         assert(out === d4) else $error("sel = 4 failed");
77
78         sel = 5; #10
79         assert(out === d5) else $error("sel = 5 failed");
80
81         sel = 0; #10
82         assert(out === d0) else $error("sel = 0 failed");
83
84         sel = 2; #10
85         assert(out === d2) else $error("sel = 2 failed");
86
87         sel = 0; #10
88         assert(out === d0) else $error("sel = 0 failed");
89

```

```

90     sel = 4; #10
91     assert(out === d4) else $error("sel = 4 failed");
92
93     // ~ edge case : input changes ~
94     sel = 3; #10
95     assert(out === d3) else $error("sel = 3 failed");
96
97     d3 = 32'hF000_0000; #10
98     assert(out === 32'hF000_0000) else $error("h7 failed");
99
100    d3 = 32'h0F00_0000; #10
101    assert(out === 32'h0F00_0000) else $error("h6 failed");
102
103    d3 = 32'h00F0_0000; #10
104    assert(out === 32'h00F0_0000) else $error("h5 failed");
105
106    d3 = 32'h000F_0000; #10
107    assert(out === 32'h000F_0000) else $error("h4 failed");
108
109    d3 = 32'h0000_F000; #10
110    assert(out === 32'h0000_F000) else $error("h3 failed");
111
112    d3 = 32'h0000_0F00; #10
113    assert(out === 32'h0000_0F00) else $error("h2 failed");
114
115    d3 = 32'h0000_00F0; #10
116    assert(out === 32'h0000_00F0) else $error("h1 failed");
117
118    d3 = 32'h0000_000F; #10
119    assert(out === 32'h0000_000F) else $error("h0 failed");
120
121    // ~ default case ~
122    sel = 6;
123    while(sel < 15) begin
124        #10;
125        assert(out === (32'hFFFF_FFFF - 13)) else $error("default %d failed", sel);
126        sel = sel + 1;
127    end
128
129 end
130
131
132 endmodule

```

Listing 4: Simulation file for PC\_Reg.sv

```

1  'timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/17/2026 06:36:03 PM
7  // Design Name:
8  // Module Name: PC_Reg_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module PC_Reg_tb();
24
25     // ~~ local variables ~~
26     logic clk, reset, write;
27     logic[31:0] din, dout;
28     logic[31:0] answer;
29
30
31     // ~~ high-level module instances (and hooks) ~~
32     PC_Reg UUT (
33         .CLK(clk),
34         .PC_RST(reset),
35         .PC_WE(write),
36         .PC_DIN(din),
37         .PC_COUNT(dout)
38     );
39
40     // ~~ action : CLK (10ns period) ~~
41     always begin
42         #5;
43         clk <= !clk;
44     end
45

```

```

46 // ~~ action : Data ~~
47 initial begin
48
49     // ~ init ~
50     clk = 1; reset = 0; write = 0;
51     reset = 1; #10; // zero out register
52
53     // ~ test cases ~
54     answer = 32'hDEAD_BEEF;
55     din = answer;
56
57     // ~ TC1 : write ~
58     reset = 0; #10;
59     assert(dout == 0) else $error("TC1: init failed");
60     write = 1; #10;
61     assert(dout == answer) else $error("TC1: write failed");
62
63     // ~ TC2: reset after write ~
64     reset = 1; #10;
65     assert(dout == 0) else $error("TC2: reset failed");
66
67     // ~ TC3: reset priority over write ~
68     write = 1; reset = 1; #10;
69     assert(dout == 0) else $error("TC3: reset failed");
70
71
72     // ~ TC4 : write within range ~
73     write = 1; reset = 0; #10;
74     answer = 32'h0246_8ACE;
75     for (int i = 0; i < 8; i++) begin
76         din = answer; #10;
77         assert(dout == answer) else $error("TC4: write failed");
78         answer = answer << 4;
79     end
80
81     // ~ TC5 : Rollover ~
82     din = 32'hFFFF_FFFF; #10;
83     assert(dout == din) else $error("TC5: write failed");
84     din = din + 1; #10;
85     assert(dout == 0) else $error("TC5: rollover failed");
86
87
88
89
90 end
91
92
93 endmodule

```

## SystemVerilog Code

Listing 5: PC\_TestBlock.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/17/2026 07:34:25 PM
7  // Design Name:
8  // Module Name: PC_TestBlock
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module PC_TestBlock(
24     input logic PC_RST, PC_WE, CLK,
25     input logic[2:0] PC_SEL,
26     input logic[31:0] JALR, BRANCH, JAL, MTVEC, MEPC,
27     output logic[31:0] PC_COUNT, PC_PLUS_FOUR
28 );
29
30 // ~~~~ inits and wires ~~~~
31 logic[31:0] MUX_0_WIRE;
32
33 // ~~~~ instances ~~~~
34 Mux_6N #(32) Mux(
35     .D0(PC_PLUS_FOUR),
36     .D1(JALR),

```

```

37     .D2(BRANCH),
38     .D3(JAL),
39     .D4(MTVEC),
40     .D5(MEPC),
41     .S(PC_SEL),
42     .Y(MUX_O_WIRE)
43 );
44
45
46 PC_Reg PC(
47     .CLK(CLK),
48     .PC_RST(PC_RST),
49     .PC_WE(PC_WE),
50     .PC_DIN(MUX_O_WIRE),
51     .PC_COUNT(PC_COUNT)
52 );
53
54 Plus4_1N #(32) P4(
55     .IN(PC_COUNT),
56     .OUT(PC_PLUS_FOUR)
57 );
58
59
60 endmodule

```

0.1.3 HW3 - Register File

Intended Behavior(s)

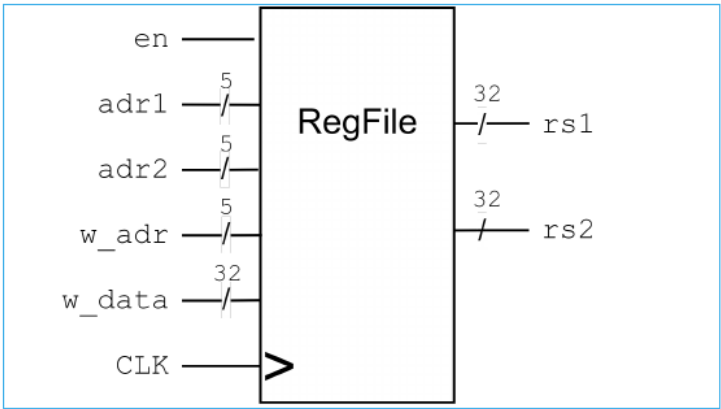


Figure 9: Top Level Register File

The RAM register file must ...

- store 32 32-bit registers
- dual asynch fetch the values at ADR1 / ADR2 and place them in RS1, RS2
- single synch write W\_DATA to W\_ADDR if EN is high.
- prevent x0 "zero" register from being overwritten.

Structural Design(s)

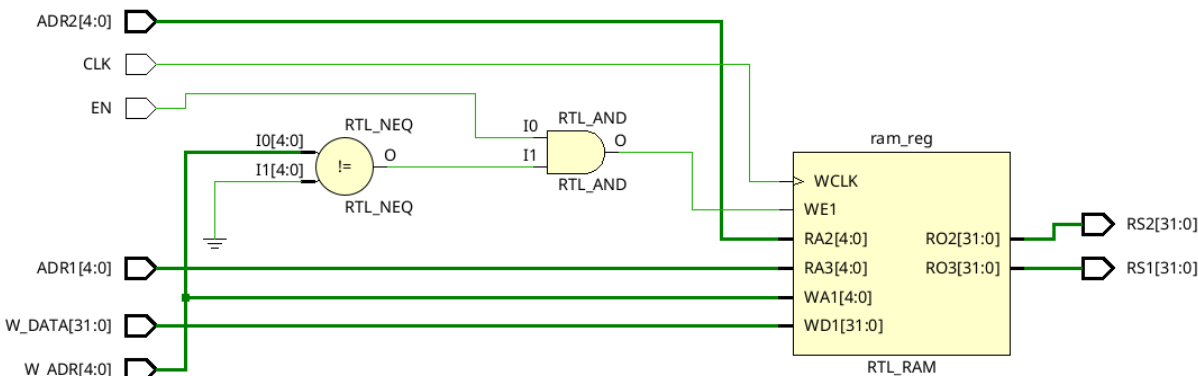


Figure 10: RTL Design of RegFile.sv

Notice we only overwrite the W\_ADDR if EN is high AND W\_ADDR is not 0 (for x0).

Synthesis Warning(s)

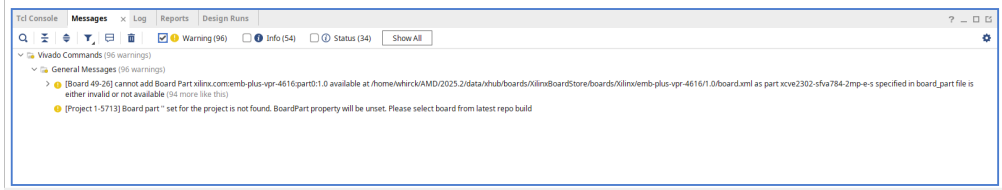


Figure 11: Synthesis Errors of RegFile.sv

Verification

All test cases passed in RegFile\_tb.sv passed in simulation along with each subcomponent test bench.

Listing 6: Verification Simulation file for RegFile.sv

```
1 `timescale 1ns / 1ps
2 //////////////////////////////////////
3 // Company:
4 // Engineer: WhisperingRock
5 //
```

```

6 // Create Date: 07/21/2026 03:42:37 PM
7 // Design Name:
8 // Module Name: RegFile_tb
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22 module RegFile_tb();
23
24 // ~~~~ init local vars ~~~~
25 logic enable, clk; // input
26 logic[4:0] addr1, addr2, w_addr; // input
27 logic[31:0] w_val; // input
28 logic[31:0] out1, out2; // output
29
30
31 // ~~~~ instances ~~~~
32 RegFile UUT(
33     .EN(enable),
34     .CLK(clk),
35     .ADR1(addr1),
36     .ADR2(addr2),
37     .W_ADR(w_addr),
38     .W_DATA(w_val),
39     .RS1(out1),
40     .RS2(out2)
41 );
42
43 // ~~ predefined tasks ~~
44 task fill_ram(input int bias);
45     enable = 1;
46     for(int i = 0; i < N_REGS; i++)
47     begin
48         w_addr = i;
49         w_val = i + bias;
50         #10;
51     end
52     enable = 0;
53 endtask
54
55 // ~~ action : CLK (10ns period) ~~
56 always begin
57     #5;
58     clk <= !clk;
59 end
60
61 // ~~~~ action : data ~~~~
62 initial begin
63
64     // ~~ init ~~
65     clk = 1; enable = 0;
66     addr1 = 0; addr2 = 0;
67     w_addr = 0; w_val = 0;
68     #10;
69     fill_ram(0);
70     #10
71
72     // ~~ TC1 : same asynch fetch ~~
73     for(int i = 0; i < 32; i++)
74     begin
75         addr1 = i;
76         addr2 = i;
77         #10;
78         assert(out1 == i) else $error("TC1: rs1 failed");
79         assert(out2 == i) else $error("TC1: rs2 failed");
80     end
81
82     // ~~ TC2 : prevent write to x0 ~~
83     addr1 = 0; enable = 1;
84     #10;
85     for(int i = 0; i < 100; i++)
86     begin
87         w_addr = 0;
88         w_val = $random();
89         #10;
90         assert(out1 == 0) else $error("TC2: x0 prevent failed");
91     end
92
93     // ~~ TC3 : write not enabled on same addr ~~
94     fill_ram(0); // cheap sanitizer
95     addr1 = 1; addr2 = 1; w_addr = 1; // vals should be 1 in reg
96     enable = 0;

```

```

97         #10;
98         for(int i = 0; i < 100; i++)
99         begin
100             w_val = $random();
101             #10;
102             assert(out1 == 1) else $error("TC3: write prevent failed");
103             assert(out2 == 1) else $error("TC3: write prevent failed");
104         end
105
106
107         // ~~ TC4 : reading the register being written to ~~
108         addr1 = 14; addr2 = 14; w_addr = 14;
109         enable = 1;
110         #10;
111         for(int i = 0; i < 100; i++)
112         begin
113             w_val = $random();
114             #10;
115             assert(out1 == w_val) else $error("TC4: r/w failed");
116             assert(out2 == w_val) else $error("TC4: r/w failed");
117         end
118
119     end
120
121 endmodule
122

```

## SystemVerilog Code

Listing 7: RegFile.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer: WhisperingRock
5  //
6  // Create Date: 07/21/2026 03:41:49 PM
7  // Design Name:
8  // Module Name: RegFile
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //         A register file is a block of RAM, not ROM,
14 //         that contains the values of all 32 32-bit
15 //         registers x0:x31, with x0 held constant to
16 //         zero.
17 //
18 //         This register is a dual asynch fetch s.t.
19 //         the values at ADR1 and ADR2 are fetched
20 //         and forwarded immediately to RS1, RS2.
21 //
22 //         This register is single synch write s.t.
23 //         if en is high, W_DATA will overwrite the
24 //         value in W_ADR (excluding x0)
25 //
26 // Dependencies:
27 //
28 // Revision:
29 // Revision 0.01 - File Created
30 // Additional Comments:
31 //
32 ///////////////////////////////////////////////////////////////////
33 parameter BITWIDTH      = 32;
34
35 parameter N_REGS        = 32;
36 parameter REG_BUSWIDTH  = 5;
37
38 module RegFile(
39     input EN, CLK,
40     input[REG_BUSWIDTH-1:0] ADR1, ADR2,    // asynch fetch addr(s)
41     input[REG_BUSWIDTH-1:0] W_ADR,        // synch write addr
42     input[BITWIDTH-1:0] W_DATA,          // synch write value
43     output[BITWIDTH-1:0] RS1, RS2        // asynch fetch'd values
44 );
45
46
47 // ~~~~ alloc mem ~~~~
48 logic [BITWIDTH-1:0] ram [0:N_REGS-1];    // memory uses big-endian
49
50 // ~~~~ init mem ~~~~
51 initial
52 begin
53     for(int i = 0; i < N_REGS; i++)
54     begin
55         ram[i] = 0;
56     end
57 end
58

```

```

59 // ~~~~ asynch fetch ~~~~
60 assign RS1 = ram[ADR1];
61 assign RS2 = ram[ADR2];
62
63
64 // ~~~~ synch write ~~~~
65 always_ff0(posedge CLK) // changes on clk demand sequential logic (use nonblocking)
66 begin
67     if(EN & W_ADR != 0)
68     begin
69         ram[W_ADR] <= W_DATA;
70     end
71 end
72
73 endmodule

```

0.1.4 HW4 - ALU + ImmGen

ALU

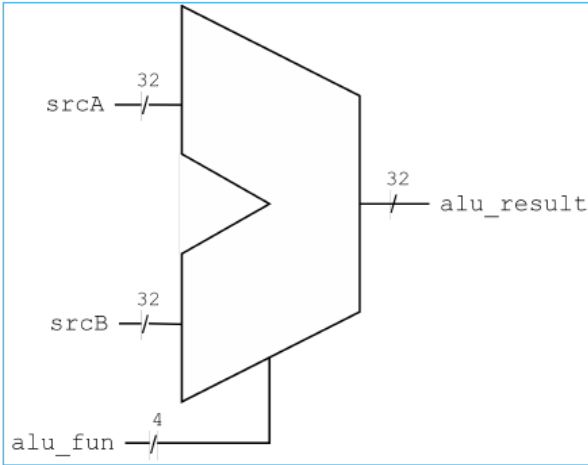


Figure 12: Top Level ALU

The ALU is designed to perform the arithmetic and logic operations required by the RISC-V CPU. Note the 11 we implemented here.

| ALU_FUN | Instruction |
|---------|-------------|
| 0000    | ADD         |
| 1000    | SUB         |
| 0110    | OR          |
| 0111    | AND         |
| 0100    | XOR         |
| 0101    | SRL         |

| ALU_FUN | Instruction |
|---------|-------------|
| 0001    | SLL         |
| 1101    | SRA         |
| 0010    | SLT         |
| 0011    | SLTU        |
| 1001    | LUI-COPY    |

Figure 13: Instruction Operations of the ALU

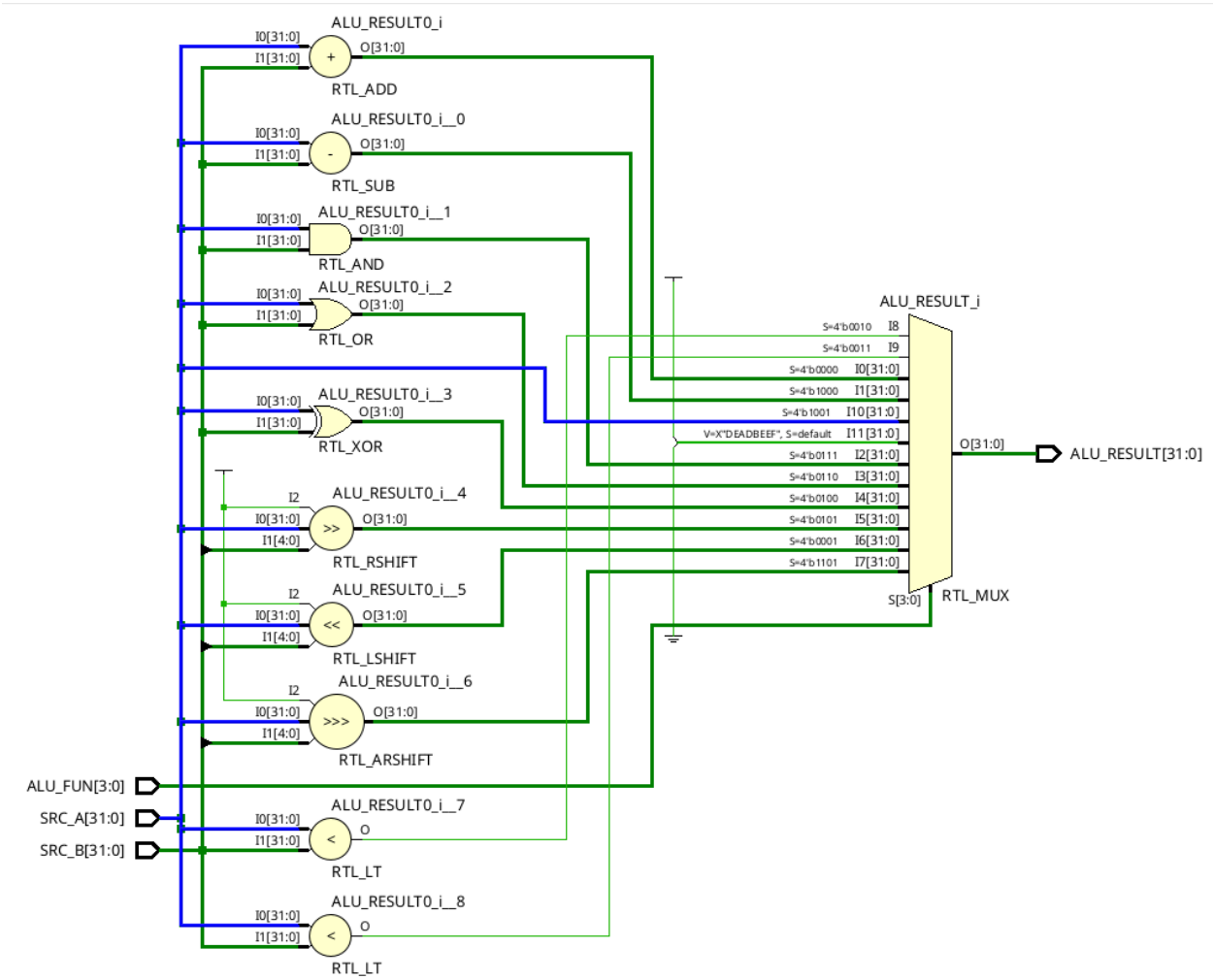


Figure 14: RTL Design of the ALU

| Function       | ALU_FUN | Arg A       | Arg B       | Expected Result |
|----------------|---------|-------------|-------------|-----------------|
| ADD            | 0x0000  | 0xA50F_96C3 | 0x5AF0_693C | 0xFFFF_FFFF     |
|                |         | 0x8410_5F21 | 0x7B10_5FDE | 0xFF20_BEFF     |
|                |         | 0xFFFF_FFFF | 0x0000_0001 | 0x0000_0000     |
| SUB            | 0x1000  | 0x0000_0000 | 0x0000_0001 | 0xFFFF_FFFF     |
|                |         | 0xAA80_6355 | 0x5501_62AA | 0x557F_00AB     |
|                |         | 0x5501_62AA | 0xAA80_6355 | 0xAA80_FF55     |
| AND            | 0x0111  | 0xA55A_00FF | 0x5A5A_FFFF | 0x005A_00FF     |
|                |         | 0xC3C3_F966 | 0xFF66_9F5A | 0xC342_9942     |
| OR             | 0x0110  | 0x9A9A_C300 | 0x65A3_CC0F | 0xFFBB_CF0F     |
|                |         | 0xC3C3_F966 | 0xFF66_9F5A | 0xFFE7_FF7E     |
| XOR            | 0x0100  | 0xAA55_00FF | 0x5AA5_0FF0 | 0xF0F0_0F0F     |
|                |         | 0xA5A5_6C6C | 0xFF00_C6FF | 0x5AA5_AA93     |
| SRL            | 0x0101  | 0x805A_6CF3 | 0x0000_0010 | 0x0000_805A     |
|                |         | 0x705A_6CF3 | 0x0000_0005 | 0x0382_D367     |
|                |         | 0x805A_6CF3 | 0x0000_0000 | 0x805A_6CF3     |
|                |         | 0x805A_6CF3 | 0x0000_0100 | 0x805A_6CF3     |
| SLL            | 0x0001  | 0x805A_6CF3 | 0x0000_0010 | 0x6CF3_0000     |
|                |         | 0x805A_6CF3 | 0x0000_0005 | 0x0B4D_9E60     |
|                |         | 0x805A_6CF3 | 0x0000_0100 | 0x805A_6CF3     |
| SRA            | 0x1101  | 0x805A_6CF3 | 0x0000_0010 | 0xFFFF_805A     |
|                |         | 0x705A_6CF3 | 0x0000_0005 | 0x0382_D367     |
|                |         | 0x805A_6CF3 | 0x0000_0000 | 0x805A_6CF3     |
|                |         | 0x805A_6CF3 | 0x0000_0100 | 0x805A_6CF3     |
| SLT            | 0x0010  | 0x7FFF_FFFF | 0x8000_0000 | 0x0000_0000     |
|                |         | 0x8000_0000 | 0x0000_0001 | 0x0000_0001     |
|                |         | 0x0000_0000 | 0x0000_0000 | 0x0000_0000     |
|                |         | 0x5555_5555 | 0x5555_5555 | 0x0000_0000     |
| SLTU           | 0x0011  | 0x7FFF_FFFF | 0x8000_0000 | 0x0000_0001     |
|                |         | 0x8000_0000 | 0x0000_0001 | 0x0000_0000     |
|                |         | 0x0000_0000 | 0x0000_0000 | 0x0000_0000     |
|                |         | 0x55AA_55AA | 0x55AA_55AA | 0x0000_0000     |
| LUI COPY (MV?) | 0x1001  | 0x0123_4567 | 0x7654_3210 | 0x0123_4567     |
|                |         | 0xFEDC_BA98 | 0x89AB_CDEF | 0xFEDC_BA98     |

Table 2: ALU Required Test Cases for Verification

Listing 8: Verification testing for ALU: ALU\_tb.sv

```
1 `timescale 1ns / 1ps
2 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 07/24/2026 10:13:51 PM
7 // Design Name:
8 // Module Name: ALU_tb
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////
21
22
23 module ALU_tb();
24
25     // ~~~~ init local vars ~~~~
26     logic[4:0] sel;
27     logic[31:0] a, b;
28     logic[31:0] result;
29
30     logic[31:0] testcase; // testing var
31
32     // ~~~~ instances ~~~~
33     ALU uut
34     (
35         .ALU_FUN(sel),
36         .SRC_A(a),
37         .SRC_B(b),
38         .ALU_RESULT(result)
39     );
40
41     // ~~~~ testing ~~~~
```

```

42     initial
43     begin
44
45         // ~~~~ ADD ~~~~
46         sel = 4'b0000;
47
48         // ~~ TC1 : ADD1 ~~
49         #5;
50         a = 32'hA50F_96C3;
51         b = 32'h5AF0_693C;
52         testcase = 1;
53         #5;
54         assert(result === 32'hFFFF_FFFF) else $error("TC1 : ADD failed");
55
56         // ~~ TC2 : ADD2 ~~
57         #5;
58         a = 32'h8410_5F21;
59         b = 32'h7B10_5FDE;
60         testcase++;
61         #5;
62         assert(result === 32'hFF20_BEFF) else $error("TC2 : ADD failed");
63
64         // ~~ TC3 : ADD3 ~~
65         #5;
66         a = 32'hFFFF_FFFF;
67         b = 32'h0000_0001;
68         testcase++;
69         #5;
70         assert(result === 32'h0000_0000) else $error("TC3 : ADD failed");
71
72
73
74         // ~~~~ SUB ~~~~
75         sel = 4'b1000;
76
77         // ~~ TC4 : SUB4 ~~
78         #5;
79         a = 32'h0000_0000;
80         b = 32'h0000_0001;
81         testcase++;
82         #5;
83         assert(result === 32'hFFFF_FFFF) else $error("TC4 : SUB failed");
84
85         // ~~ TC5 : SUB5 ~~
86         #5;
87         a = 32'hAA80_6355;
88         b = 32'h5501_62AA;
89         testcase++;
90         #5;
91         assert(result === 32'h557F_00AB) else $error("TC5 : SUB failed");
92
93         // ~~ TC6 : SUB6 ~~
94         #5;
95         a = 32'h5501_62AA;
96         b = 32'hAA80_6355;
97         testcase++;
98         #5;
99         assert(result === 32'hAA80_FF55) else $error("TC6 : SUB failed");
100
101
102
103         // ~~~~ AND ~~~~
104         sel = 4'b0111;
105
106         // ~~ TC7 : AND7 ~~
107         #5;
108         a = 32'hA55A_00FF;
109         b = 32'h5A5A_FFFF;
110         testcase++;
111         #5;
112         assert(result === 32'h005A_00FF) else $error("TC7 : AND failed");
113
114         // ~~ TC8 : AND8 ~~
115         #5;
116         a = 32'hC3C3_F966;
117         b = 32'hFF66_9F5A;
118         testcase++;
119         #5;
120         assert(result === 32'hC342_9942) else $error("TC8 : AND failed");
121
122
123
124         // ~~~~ OR ~~~~
125         sel = 4'b0110;
126
127         // ~~ TC9 : OR ~~
128         #5;
129         a = 32'h9A9A_C300;
130         b = 32'h65A3_CC0F;
131         testcase++;
132         #5;
133         assert(result === 32'hFFBB_CF0F) else $error("TC9 : OR failed");

```

```

133
134 // ~~ TC10 ~~
135 #5;
136 a = 32'hC3C3_F966;
137 b = 32'hFF66_9F5A;
138 testcase++;
139 #5;
140 assert(result === 32'hFFE7_FF7E) else $error("TC10 : OR failed");
141
142
143 // ~~~~ XOR ~~~~
144 sel = 4'b0100;
145
146 // ~~ TC11 ~~
147 #5;
148 a = 32'hAA55_00FF;
149 b = 32'h5AA5_0FF0;
150 testcase++;
151 #5;
152 assert(result === 32'hF0F0_0F0F) else $error("TC11 : XOR failed");
153
154 // ~~ TC12 ~~
155 #5;
156 a = 32'hA5A5_6C6C;
157 b = 32'hFF00_C6FF;
158 testcase++;
159 #5;
160 assert(result === 32'h5AA5_AA93) else $error("TC12 : XOR failed");
161
162
163 // ~~~~ SRL ~~~~
164 sel = 4'b0101;
165
166 // ~~ TC13 ~~
167 #5;
168 a = 32'h805A_6CF3;
169 b = 32'h0000_0010;
170 testcase++;
171 #5;
172 assert(result === 32'h0000_805A) else $error("TC13 : SRL failed");
173
174 // ~~ TC14 ~~
175 #5;
176 a = 32'h705A_6CF3;
177 b = 32'h0000_0005;
178 testcase++;
179 #5;
180 assert(result === 32'h0382_D367) else $error("TC14 : SRL failed");
181
182 // ~~ TC15 ~~
183 #5;
184 a = 32'h805A_6CF3;
185 b = 32'h0000_0000;
186 testcase++;
187 #5;
188 assert(result === 32'h805A_6CF3) else $error("TC15 : SRL failed");
189
190 // ~~ TC16 ~~
191 #5;
192 a = 32'h805A_6CF3;
193 b = 32'h0000_0100;
194 testcase++;
195 #5;
196 assert(result === 32'h805A_6CF3) else $error("TC16 : SRL failed");
197
198
199 // ~~~~ SLL ~~~~
200 sel = 4'b0001;
201
202 // ~~ TC17 ~~
203 #5;
204 a = 32'h805A_6CF3;
205 b = 32'h0000_0010;
206 testcase++;
207 #5;
208 assert(result === 32'h6CF3_0000) else $error("TC17 : SLL failed");
209
210 // ~~ TC18 ~~
211 #5;
212 a = 32'h805A_6CF3;
213 b = 32'h0000_0005;
214 testcase++;
215 #5;
216 assert(result === 32'h0B4D_9E60) else $error("TC18 : SLL failed");
217
218 // ~~ TC19 ~~
219 #5;
220 a = 32'h805A_6CF3;
221 b = 32'h0000_0100;
222
223

```

```

224     testcase++;
225     #5;
226     assert(result === 32'h805A_6CF3) else $error("TC19 : SLL failed");
227
228
229
230     // ~~~~ SRA ~~~~
231     sel = 4'b1101;
232
233     // ~~ TC20 ~~
234     #5;
235     a = 32'h805A_6CF3;
236     b = 32'h0000_0010;
237     testcase++;
238     #5;
239     assert(result === 32'hFFFF_805A) else $error("TC20 : SRA failed");
240
241     // ~~ TC21 ~~
242     #5;
243     a = 32'h705A_6CF3;
244     b = 32'h0000_0005;
245     testcase++;
246     #5;
247     assert(result === 32'h0382_D367) else $error("TC21 : SRA failed");
248
249     // ~~ TC22 ~~
250     #5;
251     a = 32'h805A_6CF3;
252     b = 32'h0000_0000;
253     testcase++;
254     #5;
255     assert(result === 32'h805A_6CF3) else $error("TC22 : SRA failed");
256
257     // ~~ TC23 ~~
258     #5;
259     a = 32'h805A_6CF3;
260     b = 32'h0000_0100;
261     testcase++;
262     #5;
263     assert(result === 32'h805A_6CF3) else $error("TC23 : SRA failed");
264
265
266
267
268     // ~~~~ SLT ~~~~
269     sel = 4'b0010;
270
271     // ~~ TC24 ~~
272     #5;
273     a = 32'h7FFF_FFFF;
274     b = 32'h8000_0000;
275     testcase++;
276     #5;
277     assert(result === 32'h0000_0000) else $error("TC24 : SLT failed");
278
279     // ~~ TC25 ~~
280     #5;
281     a = 32'h8000_0000;
282     b = 32'h0000_0001;
283     testcase++;
284     #5;
285     assert(result === 32'h0000_0001) else $error("TC25 : SLT failed");
286
287     // ~~ TC26 ~~
288     #5;
289     a = 32'h0000_0000;
290     b = 32'h0000_0000;
291     testcase++;
292     #5;
293     assert(result === 32'h0000_0000) else $error("TC26 : SLT failed");
294
295     // ~~ TC27 ~~
296     #5;
297     a = 32'h5555_5555;
298     b = 32'h5555_5555;
299     testcase++;
300     #5;
301     assert(result === 32'h0000_0000) else $error("TC27 : SLT failed");
302
303
304     // ~~~~ SLTU ~~~~
305     sel = 4'b0011;
306
307     // ~~ TC28 ~~
308     #5;
309     a = 32'h7FFF_FFFF;
310     b = 32'h8000_0000;
311     testcase++;
312     #5;
313     assert(result === 32'h0000_0001) else $error("TC28 : SLTU failed");
314

```

```

315 // ~~ TC29 ~~
316 #5;
317 a = 32'h8000_0000;
318 b = 32'h0000_0001;
319 testcase++;
320 #5;
321 assert(result === 32'h0000_0000) else $error("TC29 : SLTU failed");
322
323 // ~~ TC30 ~~
324 #5;
325 a = 32'h0000_0000;
326 b = 32'h0000_0000;
327 testcase++;
328 #5;
329 assert(result === 32'h0000_0000) else $error("TC30 : SLTU failed");
330
331 // ~~ TC31 ~~
332 #5;
333 a = 32'h55AA_55AA;
334 b = 32'h55AA_55AA;
335 testcase++;
336 #5;
337 assert(result === 32'h0000_0000) else $error("TC31 : SLTU failed");
338
339
340 // ~~~~ LUI-COPY (MV?) ~~~~
341 sel = 4'b1001;
342
343 // ~~ TC32 ~~
344 #5;
345 a = 32'h0123_4567;
346 b = 32'h7654_3210;
347 testcase++;
348 #5;
349 assert(result === 32'h0123_4567) else $error("TC32 : LUI-COPY failed");
350
351 // ~~ TC33 ~~
352 #5;
353 a = 32'hFEDC_BA98;
354 b = 32'h89AB_CDEF;
355 testcase++;
356 #5;
357 assert(result === 32'hFEDC_BA98) else $error("TC33 : LUI-COPY failed");
358
359 end
360
361 endmodule

```

Listing 9: ALU.sv

```

1 `timescale 1ns / 1ps
2 ///////////////////////////////////////////////////////////////////
3 // Company:
4 // Engineer: WhisperingRock
5 //
6 // Create Date: 07/24/2026 09:28:51 PM
7 // Design Name:
8 // Module Name: ALU
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //      A Purely combinational operation execution(er).
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21 //parameter BITWIDTH      = 32;
22 parameter FUNWIDTH        = 4;
23
24
25 module ALU(
26     input logic [FUNWIDTH-1:0] ALU_FUN,
27     input logic [BITWIDTH-1:0] SRC_A, SRC_B,
28     output logic[BITWIDTH-1:0] ALU_RESULT
29 );
30
31 // ~~ local vars ~~
32 logic[BITWIDTH-1:0] ERR = 32'hDEAD_BEEF;
33
34 always_comb
35 begin
36     case(ALU_FUN)
37         // ~~ arithmetic operators~~
38         4'b0000: begin ALU_RESULT = SRC_A + SRC_B; end // ADD
39         4'b1000: begin ALU_RESULT = SRC_A - SRC_B; end // SUB
40
41         // ~~ bitwise operators ~~

```

```
42      4'b0111: begin ALU_RESULT = SRC_A & SRC_B; end // AND
43      4'b0110: begin ALU_RESULT = SRC_A | SRC_B; end // OR
44      4'b0100: begin ALU_RESULT = SRC_A ^ SRC_B; end // XOR
45
46      // ~~ shift operators ~~
47      // Note : shifting only uses the 5 LSBs of SRC_B (see manual)
48      // Note : arithmetic shifting has its own systemverilog operator
49      4'b0101: begin ALU_RESULT = SRC_A >> SRC_B[4:0]; end // SRL
50      4'b0001: begin ALU_RESULT = SRC_A << SRC_B[4:0]; end // SLL
51      4'b1101: begin ALU_RESULT = $signed(SRC_A) >>> $signed(SRC_B[4:0]); end // SRA
52
53      // ~~ set operators ~~
54      4'b0010: begin ALU_RESULT = $signed(SRC_A) < $signed(SRC_B); end // SLT
55      4'b0011: begin ALU_RESULT = SRC_A < SRC_B; end // SLTU
56
57      // ~~ copy operator ~~
58      4'b1001: begin ALU_RESULT = SRC_A; end // LUI-COPY (MV?)
59
60      // ~~ error ~~
61      default: begin ALU_RESULT = ERR; end
62  endcase
63  end
64
65  endmodule
```

Immediate Generator

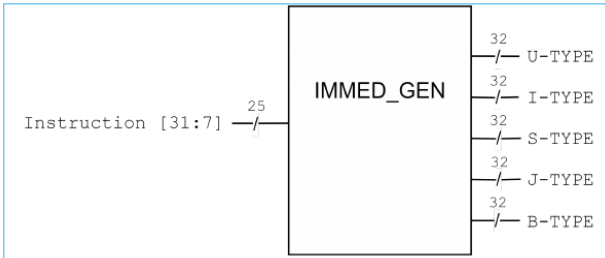


Figure 15: Top Level ImmedGen

The Immediate Generator is a feed through bit parser for different assembly instructions, continuously out-putting all types per instruction.

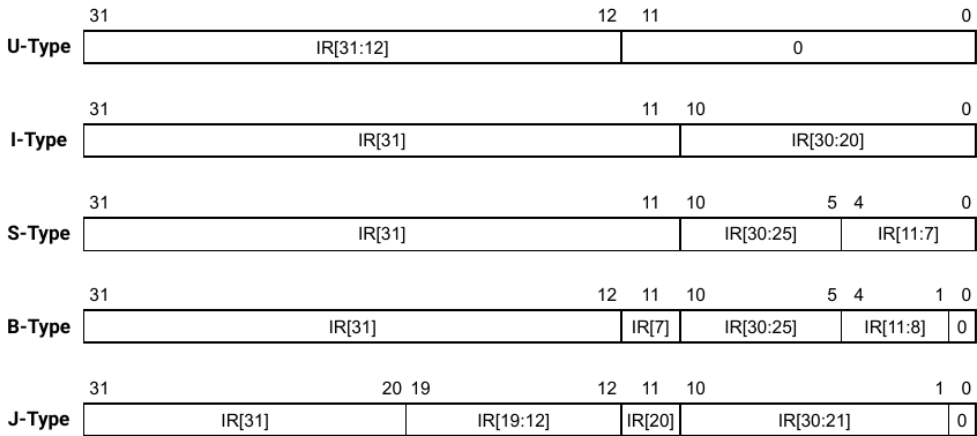


Figure 16: Instruction types of the Immediate Generator

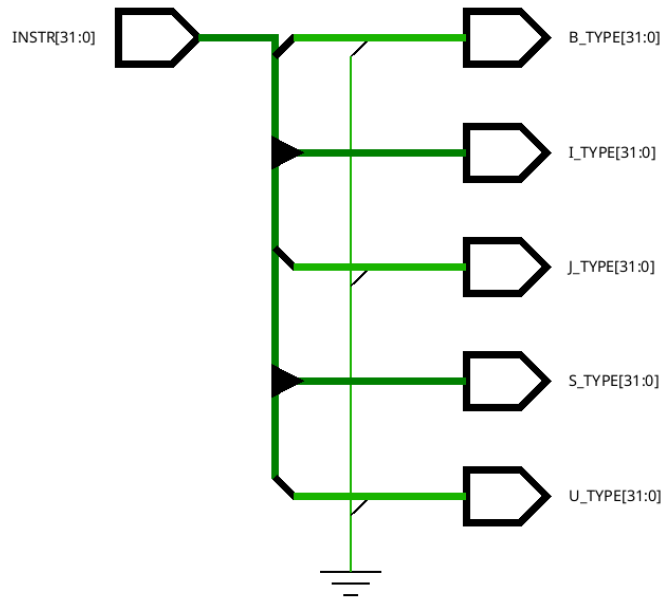


Figure 17: RTL Design of the ImmedGen

Listing 10: Verification testing for ImmedGen: ImmedGen\_tb.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/25/2026 11:41:08 AM
7  // Design Name:
8  // Module Name: ImmedGen_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module ImmedGen_tb();
24
25     // ~~~~ init local vars ~~~~
26     logic[31:0] instr;
27     logic[31:0] u, i, s, b, j;
28
29     logic[31:0] testcase; // testing var
30
31     // ~~~~ instances ~~~~
32     ImmedGen UUT
33     (
34         .INSTR(instr),
35         .U_TYPE(u),
36         .I_TYPE(i),
37         .S_TYPE(s),
38         .J_TYPE(j),
39         .B_TYPE(b)
40     );
41
42     // ~~~~ testing ~~~~
43     initial
44     begin
45
46         testcase = 1;
47
48         // ~~ TC1 : all zero ~~
49         #5;
50         testcase++;
51         instr = 0;
52         #5;
53         assert(u === 0) else $error("TC1 : U failed");
54         assert(i === 0) else $error("TC1 : I failed");
55         assert(s === 0) else $error("TC1 : S failed");
56         assert(b === 0) else $error("TC1 : B failed");
57         assert(j === 0) else $error("TC1 : J failed");
58
59         // ~~ TC2 : all one ~~
60

```

```

61         #5;
62         testcase++;
63         instr = 32'hFFFF_FFFF;
64         #5;
65         assert(u === 32'hFFFF_F000) else $error("TC2 : U failed");
66         assert(i === 32'hFFFF_FFFF) else $error("TC2 : I failed");
67         assert(s === 32'hFFFF_FFFF) else $error("TC2 : S failed");
68         assert(b === 32'hFFFF_FFFE) else $error("TC2 : B failed");
69         assert(j === 32'hFFFF_FFFE) else $error("TC2 : J failed");
70
71
72
73         // ~~ TC3 : hex1 ~~
74         #5;
75         testcase++;
76         instr = 32'h5A5A_5A5A;
77         #5;
78         assert(u === 32'h5A5A_5000) else $error("TC3 : U failed");
79         assert(i === 32'h0000_05A5) else $error("TC3 : I failed");
80         assert(s === 32'h0000_05B4) else $error("TC3 : S failed");
81         assert(b === 32'h0000_05B4) else $error("TC3 : B failed");
82         assert(j === 32'h000A_5DA4) else $error("TC3 : J failed");
83
84
85         // ~~ TC4 : hex2 ~~
86         #5;
87         testcase++;
88         instr = 32'hF0F0_F0F0;
89         #5;
90         assert(u === 32'hF0F0_F000) else $error("TC4 : U failed");
91         assert(i === 32'hFFFF_FF0F) else $error("TC4 : I failed");
92         assert(s === 32'hFFFF_FF01) else $error("TC4 : S failed");
93         assert(b === 32'hFFFF_FF00) else $error("TC4 : B failed");
94         assert(j === 32'hFFF0_FF0E) else $error("TC4 : J failed");
95
96
97         // ~~ TC5 : hex3 ~~
98         #5;
99         testcase++;
100        instr = 32'h3CA5_5A3C;
101        #5;
102        assert(u === 32'h3CA5_5000) else $error("TC5 : U failed");
103        assert(i === 32'h0000_03CA) else $error("TC5 : I failed");
104        assert(s === 32'h0000_03D4) else $error("TC5 : S failed");
105        assert(b === 32'h0000_03D4) else $error("TC5 : B failed");
106        assert(j === 32'h0005_53CA) else $error("TC5 : J failed");
107
108
109    end
110
111 endmodule

```

Listing 11: ImmedGen.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/25/2026 10:59:49 AM
7  // Design Name:
8  // Module Name: ImmedGen
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module ImmedGen(
24     input logic [BITWIDTH-1:0] INSTR,
25     output logic[BITWIDTH-1:0] U_TYPE, I_TYPE, S_TYPE, J_TYPE, B_TYPE
26 );
27
28 //~~~~~ Immediate are immediate! ~~~~~
29 assign U_TYPE = { INSTR[31:12], 12'b0 };
30
31 assign I_TYPE = { {21{INSTR[31]}}, INSTR[30:20] };
32
33 assign S_TYPE = { {21{INSTR[31]}}, INSTR[30:25], INSTR[11:7] };
34
35 assign B_TYPE =
36 {
37     {20{INSTR[31]}},

```

```
38     INSTR[7],
39     INSTR[30:25],
40     INSTR[11:8],
41     1'b0
42 };
43
44 assign J_TYPE =
45 {
46     {12{INSTR[31]}},
47     INSTR[19:12],
48     INSTR[20],
49     INSTR[30:21],
50     1'b0
51 };
52
53
54
55 endmodule
```

## 0.2 Software

### 0.2.1 SW1 - Intro to Assembly Lang

Note : Assume caller/callee does not need to preserve their respective registers in order to use them. KISS method applied for assignment 1.

#### Part 1

"Read three 16-bit unsigned values consecutively from SWITCHES (address 0x11000000), add them together, and output the result to LEDS (address 0x11000020). Assume the input values are 16-bit unsigned values. (Assume it is possible for the switches to change between each time they are read)"

Assuming the LEDS input value is limited to 16-bit unsigned as well.

#### Flowchart

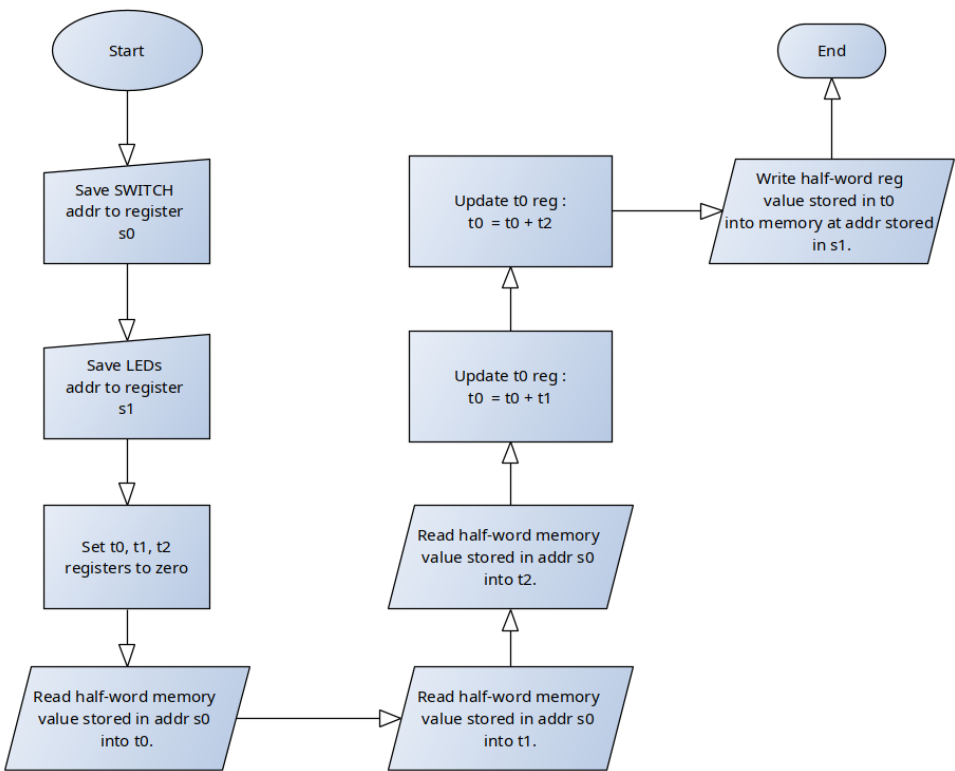


Figure 18: Flow Chart for Part 1

#### Verification Simulations

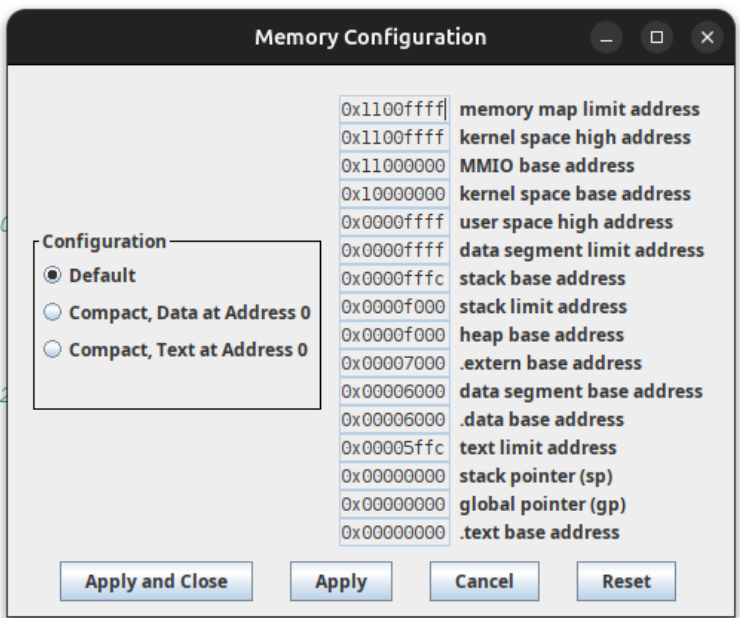


Figure 19: RARS Memory Configuration

```

20
21 #.equ SWITCHES_H,      0x11000 # upper 20 bits
22 #.equ SWITCHES_L,      0x000   # lower 12 bits
23 .equ SWITCHES_H,       0x00006 # upper 20 bits (data seg at 0x00006000)
24 .equ SWITCHES_L,       0x000   # lower 12 bits
25
26 #.equ LEDS_H,          0x11000 # upper 20 bits
27 #.equ LEDS_L,          0x020   # lower 12 bits
28 .equ LEDS_H,           0x00006 # upper 20 bits (data seg at 0x00006020)
29 .equ LEDS_L,           0x020   # lower 12 bits
30

```

Figure 20: Address Correction to work with RARS

- Test Case 1

```

33 # TC1 : Word Clip
34 .equ READ1_H,          0x00001 # switch first read (0x1234)
35 .equ READ1_L,          0x234
36
37
38 .equ READ2_H,          0x12345 # switch second read (0x5678)
39 .equ READ2_L,          0x678
40
41
42 .equ READ3_H,          0x11111 # switch third read (0x1111)
43 .equ READ3_L,          0x111
44 #   answer= 0x79BD

```

Figure 21: TC1 given inputs

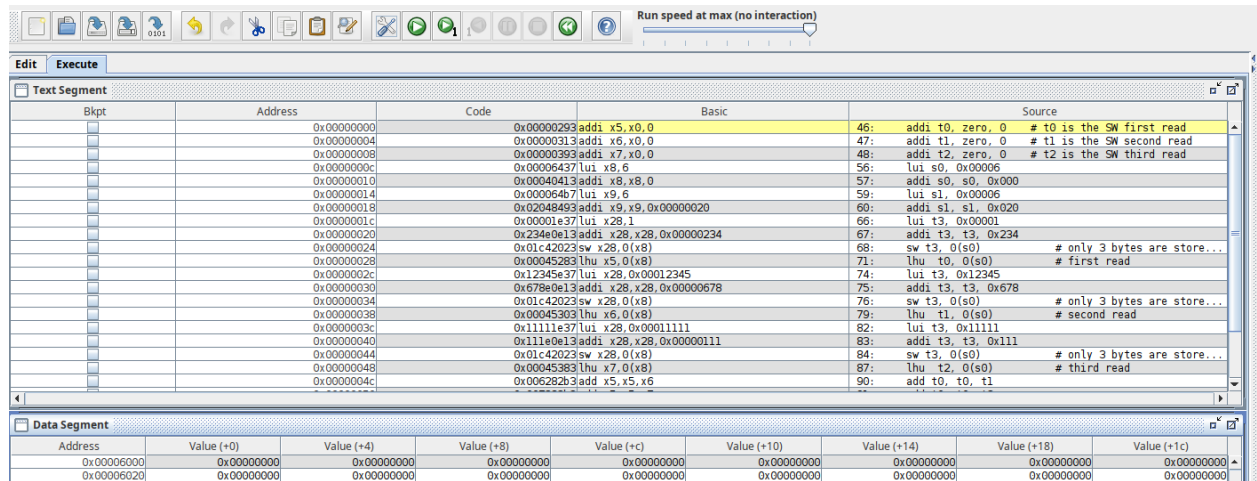


Figure 22: Data segment before executing instructions

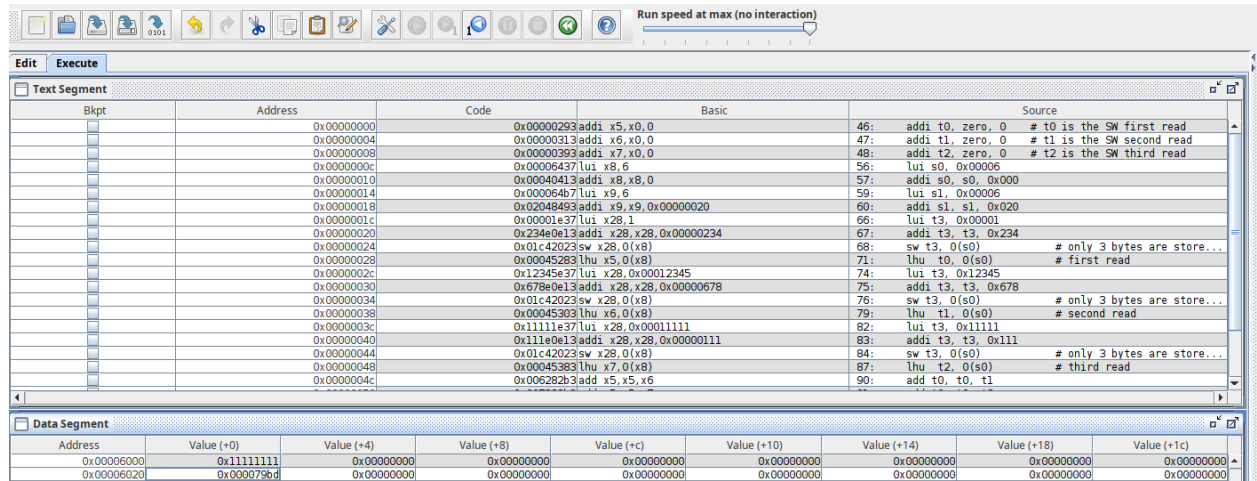


Figure 23: Data segment after executing instructions

Result at LEDS address in 23 matches the answer in 21.

- Test Case 2

```

46 # TC2 : Maxed
47 .equ READ1_H,      0x00005 # switch first read (0x5555)
48 .equ READ1_L,      0x555
49
50 .equ READ2_H,      0x00005 # switch second read (0x5555)
51 .equ READ2_L,      0x555 # sign extended
52
53
54 .equ READ3_H,      0x00005 # switch third read (0x5555)
55 .equ READ3_L,      0x555
56 # answer= 0xFFFF

```

Figure 24: TC2 given input

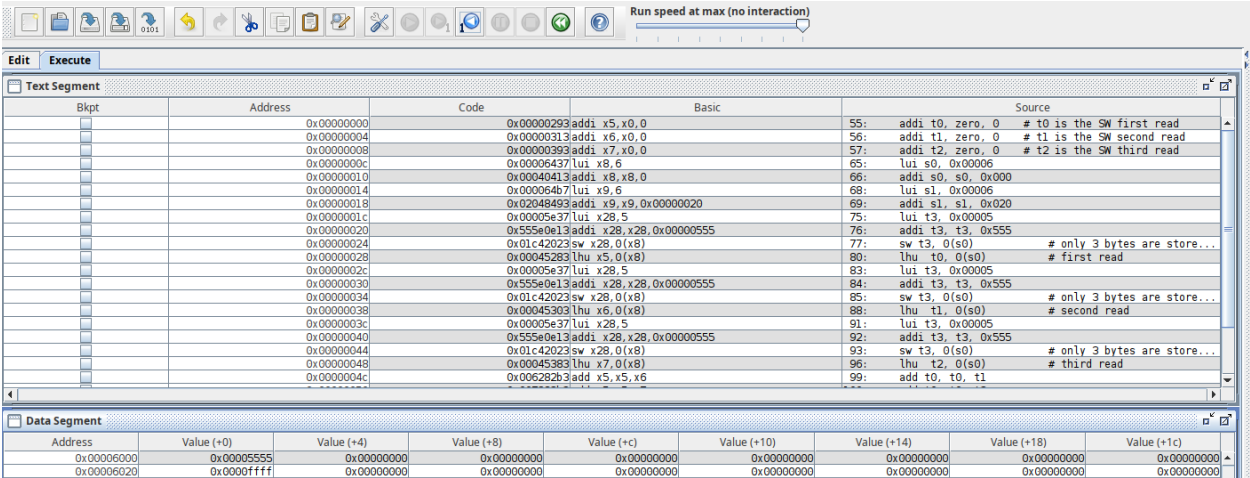


Figure 25: Data segment after executing instructions

Result at LEDS address in 25 matches the answer in 24.

- Test Case 3

```

59 # TC3 : Overflow
60 .equ READ1_H,      0x00005 # switch first read (0x5555)
61 .equ READ1_L,      0x555
62
63 .equ READ2_H,      0x00005 # switch second read (0x5555)
64 .equ READ2_L,      0x555 # sign extended
65
66
67 .equ READ3_H,      0x00005 # switch third read (0x5557)
68 .equ READ3_L,      0x557
69 # answer= 0x0001

```

Figure 26: TC3 given input

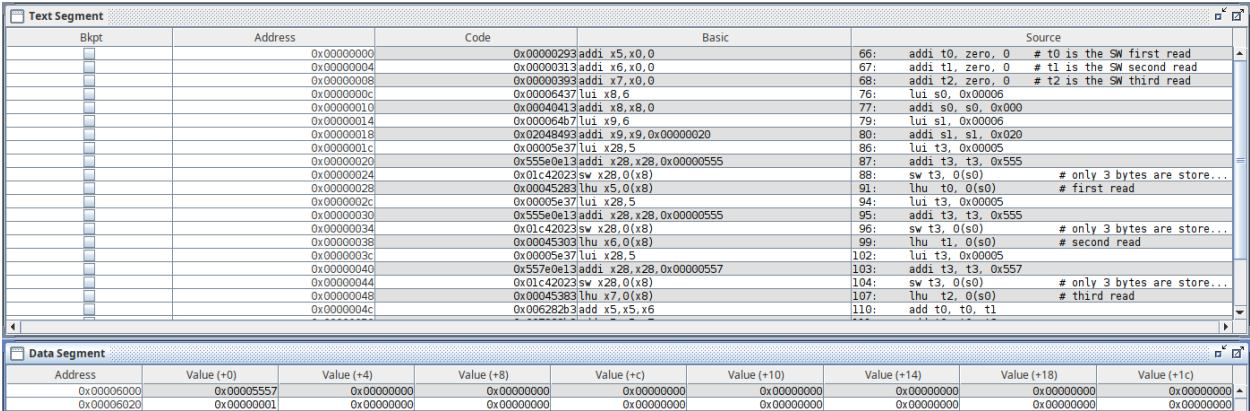


Figure 27: Data segment after executing instructions

Result at LEDS address in 27 matches the answer in 26.

- Test Case 4

```

71 # TC4 : zero
72 .equ READ1_H, 0x00000 # switch first read (0)
73 .equ READ1_L, 0x0
74
75
76 .equ READ2_H, 0x00000 # switch second read (0)
77 .equ READ2_L, 0x0 # sign extended
78
79 .equ READ3_H, 0x00000 # switch third read (0)
80 .equ READ3_L, 0x0
81 # answer= 0x0000
82

```

Figure 28: TC4 given input

[illegible]

Figure 29: Data segment after executing instructions

Result at LEDS address in 29 matches the answer in 28.

- **Test Case 5**

```

84 # TC5 : MAXXXED
85 .equ READ1_H,          0x0000F # switch first read (0xFFFF)
86 .equ READ1_L,          -1
87
88 .equ READ2_H,          0xFFFFF # switch second read (0xFFFF)
89 .equ READ2_L,          -1
90
91
92 .equ READ3_H,          0x00000 # switch third read (0)
93 .equ READ3_L,          0
94 #    answer= 0xFFFE

```

Figure 30: TC5 given input

[illegible]

Figure 31: Data segment after executing instructions

Result at LEDS address in 31 matches the answer in 30.

- **Test Case 6**

```

96
97 # TC6 : All highs into full
98 .equ READ1_H,      0x00008 # switch first read
99 .equ READ1_L,      0x000
100
101 .equ READ2_H,      0x00008 # switch second read
102 .equ READ2_L,      0x000
103
104 .equ READ3_H,      0x00008 # switch third read
105 .equ READ3_L,      0x000
106
107 #   answer= 0x8000 (0x18000 as full word though)

```

Figure 32: TC6 given input

Text Segment

| Bkpt | Address    | Code        | Basic                 | Source                                           |
|------|------------|-------------|-----------------------|--------------------------------------------------|
|      | 0x00000000 | 0x00000293  | addi x5,x0,0          | 99: addi t0, zero, 0 # t0 is the SW first read   |
|      | 0x00000004 | 0x00000313  | addi x6,x0,0          | 100: addi t1, zero, 0 # t1 is the SW second read |
|      | 0x00000008 | 0x00000363  | addi x7,t0,0          | 101: addi t2, zero, 0 # t2 is the SW third read  |
|      | 0x0000000c | 0x00006437  | lui x8,6              | 109: lui s0, 0x00006                             |
|      | 0x00000010 | 0x00040413  | addi x8,x8,0          | 110: addi s0, s0, 0x000                          |
|      | 0x00000014 | 0x000064b7  | lui x9,6              | 112: lui s1, 0x00006                             |
|      | 0x00000018 | 0x02048493  | addi x9,x9,0x00000020 | 113: addi s1, s1, 0x020                          |
|      | 0x0000001c | 0x000008e37 | lui x28,8             | 119: lui t3, 0x00008                             |
|      | 0x00000020 | 0x000e6e13  | ori x28,x28,0         | 120: ori t3, t3, 0x000                           |
|      | 0x00000024 | 0x01c42023  | sw x28,0(x8)          | 121: sw t3, 0(s0) # only 3 bytes are store...    |
|      | 0x00000028 | 0x00045283  | lhu x5,0(x8)          | 124: lhu t0, 0(s0) # first read                  |
|      | 0x0000002c | 0x000008e37 | lui x28,8             | 127: lui t3, 0x00008                             |
|      | 0x00000030 | 0x000e6e13  | ori x28,x28,0         | 128: ori t3, t3, 0x000                           |
|      | 0x00000034 | 0x01c42023  | sw x28,0(x8)          | 129: sw t3, 0(s0) # only 3 bytes are store...    |
|      | 0x00000038 | 0x00045303  | lhu x6,0(x8)          | 132: lhu t1, 0(s0) # second read                 |
|      | 0x0000003c | 0x000008e37 | lui x28,8             | 135: lui t3, 0x00008                             |
|      | 0x00000040 | 0x000e6e13  | ori x28,x28,0         | 136: ori t3, t3, 0x000                           |
|      | 0x00000044 | 0x01c42023  | sw x28,0(x8)          | 137: sw t3, 0(s0) # only 3 bytes are store...    |
|      | 0x00000048 | 0x00045383  | lhu x7,0(x8)          | 140: lhu t2, 0(s0) # third read                  |
|      | 0x0000004c | 0x006282b3  | add x5,x5,x6          | 143: add t0, t0, t1                              |

Data Segment

| Address    | Value (+0) | Value (+4) | Value (+8) | Value (+c) | Value (+10) | Value (+14) | Value (+18) | Value (+1c) |
|------------|------------|------------|------------|------------|-------------|-------------|-------------|-------------|
| 0x00006000 | 0x00008000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |
| 0x00006020 | 0x00008000 | 0x00000000 | 0x00000000 | 0x00000000 | 0x00000000  | 0x00000000  | 0x00000000  | 0x00000000  |

Figure 33: Data segment after executing instructions

Result at LEDS address in 33 matches the answer in 32.

### Assembly Source Code

Listing 12: Question 1 of SW1

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Read from the SWITCHES addr in memory three consecutive
4 #           times, summing the unsigned half-word reads and writing
5 #           to the LEDS addr.
6 #
7 # Notes :
8 #   - Assume caller/calle has no need to preserve reg values.
9 #   - KISS method for this assignment
10 #
11 #   - Split 32-bit addresses according to instr imm handling.
12
13
14
15
16 .global main
17
18 .equ SWITCHES_H, 0x11000 # upper 20 bits
19 .equ SWITCHES_L, 0x000 # lower 12 bits
20 .equ SWITCHES_H, 0x00006 # upper 20 bits (data seg at 0x00006000)
21 .equ SWITCHES_L, 0x000 # lower 12 bits
22
23 .equ LEDS_H, 0x11000 # upper 20 bits
24 .equ LEDS_L, 0x020 # lower 12 bits
25 .equ LEDS_H, 0x00006 # upper 20 bits (data seg at 0x00006020)
26 .equ LEDS_L, 0x020 # lower 12 bits
27
28 # ~~~~~ Test Cases ~~~~~
29 # TC1 : Word Clip
30 .equ READ1_H, 0x00001 # switch first read (0x1234)
31 .equ READ1_L, 0x234
32
33 .equ READ2_H, 0x12345 # switch second read (0x5678)
34 .equ READ2_L, 0x678
35
36 .equ READ3_H, 0x11111 # switch third read (0x1111)
37 .equ READ3_L, 0x111
38 # answer= 0x79BD
39
40 # TC2 : Maxed
41 .equ READ1_H, 0x00005 # switch first read (0x5555)

```

```

42 #.equ READ1_L,      0x555
43
44 #.equ READ2_H,      0x00005 # switch second read (0x5555)
45 #.equ READ2_L,      0x555 # sign extended
46
47 #.equ READ3_H,      0x00005 # switch third read (0x5555)
48 #.equ READ3_L,      0x555
49 # answer= 0xFFFF
50
51 # TC3 : Overflow
52 #.equ READ1_H,      0x00005 # switch first read (0x5555)
53 #.equ READ1_L,      0x555
54
55 #.equ READ2_H,      0x00005 # switch second read (0x5555)
56 #.equ READ2_L,      0x555 # sign extended
57
58 #.equ READ3_H,      0x00005 # switch third read (0x5557)
59 #.equ READ3_L,      0x557
60 # answer= 0x0001
61
62 # TC4 : zero
63 #.equ READ1_H,      0x00000 # switch first read (0)
64 #.equ READ1_L,      0x0
65
66 #.equ READ2_H,      0x00000 # switch second read (0)
67 #.equ READ2_L,      0x0 # sign extended
68
69 #.equ READ3_H,      0x00000 # switch third read (0)
70 #.equ READ3_L,      0x0
71 # answer= 0x0000
72
73 # TC5 : MAXXXED
74 #.equ READ1_H,      0x0000F # switch first read (0xFFFF)
75 #.equ READ1_L,      -1
76
77 #.equ READ2_H,      0xFFFFF # switch second read (0xFFFF)
78 #.equ READ2_L,      -1
79
80 #.equ READ3_H,      0x00000 # switch third read (0)
81 #.equ READ3_L,      0
82 # answer= 0xFFFE
83
84 # TC6 : All highs into full
85 .equ READ1_H,      0x00008 # switch first read
86 .equ READ1_L,      0x000
87
88 .equ READ2_H,      0x00008 # switch second read
89 .equ READ2_L,      0x000
90
91 .equ READ3_H,      0x00008 # switch third read
92 .equ READ3_L,      0x000
93 # answer= 0x8000 (0x18000 as full word though)
94 # ~~~~~
95
96 main:
97
98 # ~~~~ register init ~~~~
99 addi t0, zero, 0 # t0 is the SW first read
100 addi t1, zero, 0 # t1 is the SW second read
101 addi t2, zero, 0 # t2 is the SW third read
102 # t3 is a dummy variable for testing
103
104 # s0 contains the SWITCH addr
105 # s1 contains the LEDS addr
106
107
108 # ~~~~ load addr into registers ~~~~
109 lui s0, SWITCHES_H
110 addi s0, s0, SWITCHES_L
111
112 lui s1, LEDS_H
113 addi s1, s1, LEDS_L
114
115
116 # ~~~~ read SWITCHES and sum ~~~~
117
118 # ~~ behind the scenes : switch mem changes value ~~
119 lui t3, READ1_H
120 ori t3, t3, READ1_L
121 sw t3, 0(s0) # only 3 bytes are stored (0x234)
122 # ~~~~~
123
124 lhu t0, 0(s0) # first read
125
126 # ~~ behind the scenes : switch mem changes value ~~
127 lui t3, READ2_H
128 ori t3, t3, READ2_L
129 sw t3, 0(s0) # only 3 bytes are stored (0x678)
130 # ~~~~~
131
132 lhu t1, 0(s0) # second read

```

```
133
134 # ~~~~ behind the scenes : switch mem changes value ~~~~
135 lui t3, READ3_H
136 ori t3, t3, READ3_L
137 sw t3, 0(s0)      # only 3 bytes are stored (0x111)
138 # ~~~~~
139
140 lhu t2, 0(s0)      # third read
141
142 # ~~~~ sum values ~~~~
143 add t0, t0, t1
144 add t0, t0, t2
145
146
147 # ~~~~ Write the unsigned half word to LEDS mem addr
148 sh t0, 0(s1)
149
150
151 # ~~~~ return with exit status ~~~~
152 li a7, 10 # load system call number for exit()
153 li a0, 0  # Load the exit status code ( EXIT_SUCCESS )
154 ecall    # Make the system call
```

Part 2

"Read an input from SWITCHES (address 0x11000000). Assume the input is a 16-bit signed value in 2's complement (RC). Change the sign of the input and output the result to LEDS (address 0x11000020). The result should also be a 16-bit signed value in 2's complement (RC)."

Flowchart

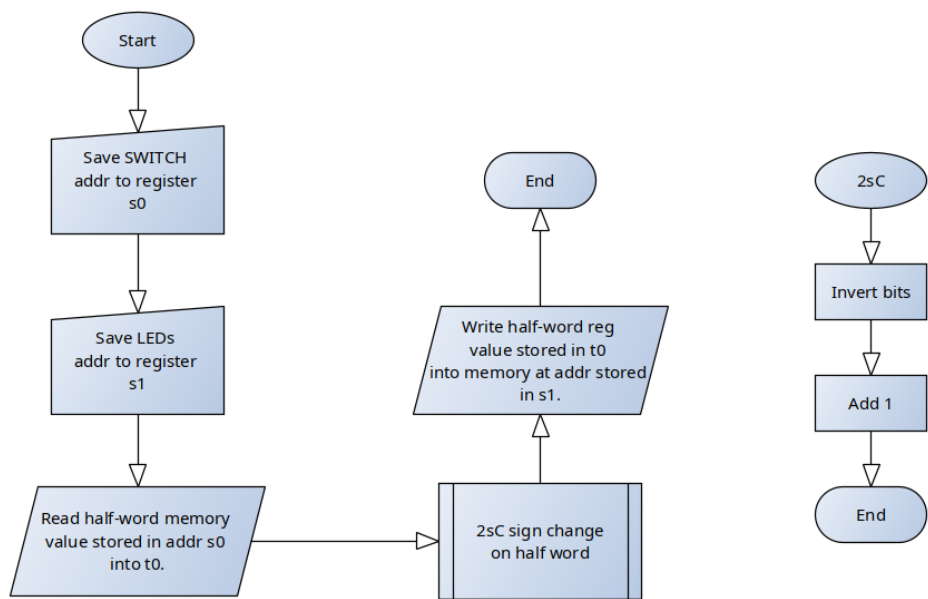


Figure 34: Flow Chart for Part 2

Verification Simulations

Test cases passed with definitions defined in code below.

```
16 # TC1 : zero
17 #A: .half 0
18 # answer= 0x0000
```

| Data Segment |            |
|--------------|------------|
| Address      | Value (+0) |
| 0x00006000   | 0x00000000 |
| 0x00006020   | 0x00000000 |

Figure 35: IO and Data Segment of TC1

```
21 # TC2 : positive
22 #A: .half 0x5
23 # answer= 0xFFFFB (-5)
```

| Data Segment |             |
|--------------|-------------|
| Address      | Value (+0)  |
| 0x00006000   | 0x00000005  |
| 0x00006020   | 0x0000ffffb |

Figure 36: IO and Data Segment of TC2

|    |                  |      |  |
|----|------------------|------|--|
| 25 | # TC3 : negative |      |  |
| 26 | #A: .half 0xFFFB | # -5 |  |
| 27 | # answer= 0x0005 | # 5  |  |
| 28 |                  |      |  |

| Data Segment |            |
|--------------|------------|
| Address      | Value (+0) |
| 0x00006000   | 0x0000ffff |
| 0x00006020   | 0x00000005 |

Figure 37: IO and Data Segment of TC3

|    |                  |          |  |
|----|------------------|----------|--|
| 30 | # TC4 : positive |          |  |
| 31 | #A: .half 0x5678 | # 22136  |  |
| 32 | # answer= 0xA988 | # -22136 |  |

| Data Segment |            |
|--------------|------------|
| Address      | Value (+0) |
| 0x00006000   | 0x00005678 |
| 0x00006020   | 0x0000a988 |

Figure 38: IO and Data Segment of TC4

|    |                  |  |  |
|----|------------------|--|--|
| 34 | # TC5 : reverse  |  |  |
| 35 | #A: .half 0xA988 |  |  |
| 36 | # answer= 0x5678 |  |  |
| 37 |                  |  |  |

| Data Segment |            |
|--------------|------------|
| Address      | Value (+0) |
| 0x00006000   | 0x0000a988 |
| 0x00006020   | 0x00005678 |

Figure 39: IO and Data Segment of TC5

|    |                         |  |  |
|----|-------------------------|--|--|
| 39 | # TC6 : negative        |  |  |
| 40 | #A : .half 0xFFFF #(-1) |  |  |
| 41 | # answer= 1             |  |  |
| 42 |                         |  |  |

| Data Segment |            |
|--------------|------------|
| Address      | Value (+0) |
| 0x00006000   | 0x0000ffff |
| 0x00006020   | 0x00000001 |

Figure 40: IO and Data Segment of TC6

|    |                         |  |  |
|----|-------------------------|--|--|
| 43 | # TC7 : negative 2      |  |  |
| 44 | #A : .half 0xFFFE #(-2) |  |  |
| 45 | # answer= 2             |  |  |
| 46 |                         |  |  |

| Data Segment |            |
|--------------|------------|
| Address      | Value (+0) |
| 0x00006000   | 0x0000fffe |
| 0x00006020   | 0x00000002 |

Figure 41: IO and Data Segment of TC7

|    |                         |                    |  |
|----|-------------------------|--------------------|--|
| 48 | # TC7 : negative 32 bit |                    |  |
| 49 | #A : .word 0x89ABCDEF   | #(0xCDEF = -12817) |  |
| 50 | # answer= 0x3211        | # 12817            |  |
| 51 |                         |                    |  |

| Data Segment |            |
|--------------|------------|
| Address      | Value (+0) |
| 0x00006000   | 0x89abcdef |
| 0x00006020   | 0x00003211 |

Figure 42: IO and Data Segment of TC8

Assembly Code

Listing 13: Question 2 of SW1

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Read from the SWITCHES addr in memory a unsigned half-word
4 #           changing the sign, then writing to the LEDS addr.
5 #
6 # Notes :
7 #     - Assume caller/callee has no need to preserve reg values.
8 #       KISS method for this assignment
9 #
10 #     - Split 32-bit addresses according to instr imm handling.
11
12 .data
13 # ~~~~~ Test Cases ~~~~~
14 # TC1 : zero
15 #A: .half 0
16 # answer= 0x0000
17
18 # TC2 : positive
19 #A: .half 0x5
20 # answer= 0xFFFB (-5)
21
22 # TC3 : negative
23 #A: .half 0xFFFB # -5
24 # answer= 0x0005 # 5
25
26 # TC4 : positive
27 #A: .half 0x5678 # 22136

```

```

28 # answer= 0xA988      # -22136
29
30 # TC5 : reverse
31 #A: .half 0xA988
32 # answer= 0x5678
33
34 # TC6 : negative
35 #A : .half 0xFFFF #(-1)
36 # answer= 1
37
38 # TC7 : negative 2
39 #A : .half 0xFFFE #(-2)
40 # answer= 2
41
42 # TC7 : negative 32 bit
43 #A : .word 0x89ABCDEF    #(0xCDEF = -12817)
44 # answer= 0x3211      # 12817
45
46 # ~~~~~
47
48
49 .global _start
50 .text
51 #.equ SWITCHES_H, 0x11000 # upper 20 bits
52 #.equ SWITCHES_L, 0x000 # lower 12 bits
53 #.equ SWITCHES_H, 0x00006 # upper 20 bits (data seg at 0x00006000)
54 #.equ SWITCHES_L, 0x000 # lower 12 bits
55
56 #.equ LEDS_H, 0x11000 # upper 20 bits
57 #.equ LEDS_L, 0x020 # lower 12 bits
58 #.equ LEDS_H, 0x00006 # upper 20 bits (data seg at 0x00006020)
59 #.equ LEDS_L, 0x020 # lower 12 bits
60
61
62
63 _start:
64
65 # ~~~~ register init ~~~~
66 addi t0, zero, 0 # t0 is the SW first read
67 # t3 is a dummy variable for testing
68
69 # s0 contains the SWITCH addr
70 # s1 contains the LEDS addr
71
72 # ~~~~ load addr into registers ~~~~
73 lui s0, SWITCHES_H
74 addi s0, s0, SWITCHES_L
75
76 lui s1, LEDS_H
77 addi s1, s1, LEDS_L
78
79
80 # ~~~~ read SWITCHES and sum ~~~~
81
82 # ~~ behind the scenes : switch value ~~
83 lw t3, A
84 sw t3, 0(s0)
85 # ~~~~~
86
87 lhu t0, 0(s0) # first read
88
89
90 # ~~~~ invert the sign (2sC) ~~~~
91 not t0, t0
92 addi t0, t0, 1
93
94 # ~~~~ Write the unsigned half word to LEDS mem addr
95 sh t0, 0(s1)
96
97
98 # ~~~~ return with exit status ~~~~
99 li a7, 10 # load system call number for exit()
100 li a0, 0 # Load the exit status code ( EXIT_SUCCESS )
101 ecall # Make the system call

```

0.2.2 SW2 - Conditional Statements

**Assignment:**  
First, create a flowchart that will perform the following behavior. Then implement the flowchart with OTTER MCU assembly language using only the instructions in the OTTER assembly manual. Make sure your code is formatted properly including comments. Use the RARS simulator to ensure the program performs as desired.

- 1. Read a 32-bit unsigned value from SWITCHES (address 0x11000000). If the input is greater than or equal to 32,768, the value is divided by 4. You can ignore any remainder. If the value is less than 32,768, the value is multiplied by 2. The result should be written to the 7 SEGMENT (address 0x11000040).
- 2. Read a 32-bit unsigned value from SWITCHES (address 0x11000000). If the input value is a multiple of 4, all of the bits should be inverted, otherwise, if the input value is odd, add 4095 and divide the result by 2, otherwise subtract 1 from the value. The result should be written to the 7 SEGMENT (address 0x11000040).

Assume the BASYS3 SWITCHES are expanded to 32 individual switches and 7 SEGMENT is expanded to 8 digits.

Figure 43: Questions to address

Part 1

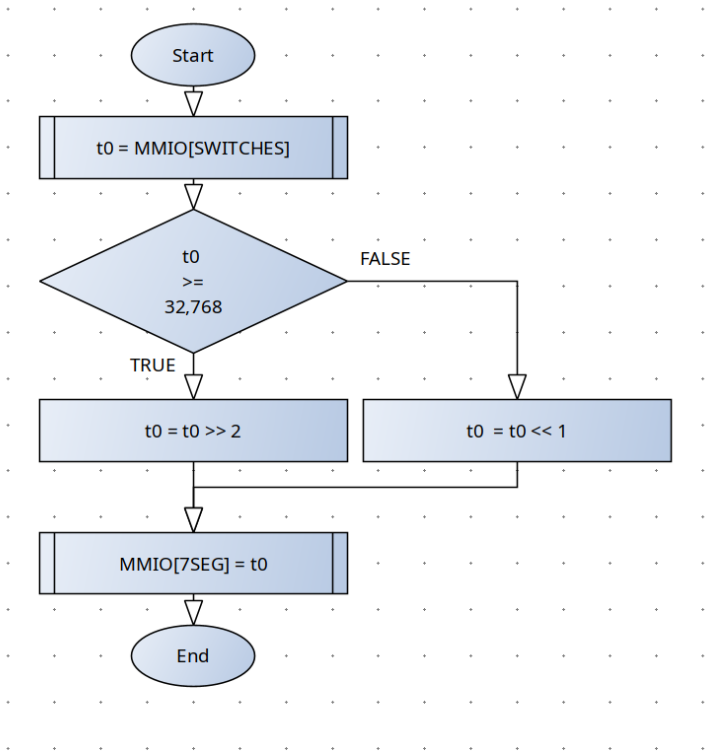


Figure 44: Flow chart for part 1

| TC# | Desc                    | Given MMIO[SWITCHES] | Correct MMIO[7SEG] | Result |
|-----|-------------------------|----------------------|--------------------|--------|
| 1   | Lowest unsigned val     | 0                    | 0                  | Passed |
| 2   | Below boundary          | 0xFFF                | 0x1FFE             | Passed |
| 3   | 1 before boundary       | 0x7FFF               | 0xFFFE             | Passed |
| 4   | On boundary             | 0x8000               | 0x2000             | Passed |
| 5   | 1 above boundary        | 0x8001               | 0x2000             | Passed |
| 6   | Above boundary          | 0x12345678           | 0x048D159E         | Passed |
| 7   | 32-bit unsigned ceiling | 0xFFFFFFFF           | 0x3FFFFFFF         | Passed |

Table 3: Test cases for part 1

Listing 14: sw2\_p1.asm

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Conditionals, branching, jumping
4 # Input :
5 #     SWITCHES MMIO address to 32-bit unsigned value
6 #     7SEG MMIO address to 8-digits, each of 4-bits
7 #     constant CONST
8 #
9 # Output :
10 #     Write to 7SEG addr
11
12
13 .data
14 # ~~~~ Test Cases ~~~~
15
16 # ~~ TC1 : Zero ~~
17 #TC:      .word 0
18 # answer = 0 (verified)
19
20 # ~~ TC2 : Under ~~
21 #TC:      .word 0xFFFF
22 # answer = 0x1FFE (verified)
23
24 # ~~ TC3 : One before ~~
25 #TC:      .word 0x07FFF
26 # answer = 0xFFFE (verified)
27
28 # ~~ TC4 : On boundary ~~
29 #TC:      .word 0x08000
30 # answer = 0x2000 (verified)
31
32 # ~~ TC5 : Above boundary ~~
33 #TC:      .word 0x08001
34 # answer = 0x2000 (verified)
35
36 # ~~ TC6 : Above boundary 2 ~~
37 #TC:      .word 0x12345678
38 # answer = 0x048D159E (verified)
39
40 # ~~ TC7 : 32-bit unsigned ceiling ~~
41 #TC:      .word 0xFFFFFFFF
42 # answer = 0x3FFFFFFF (verified)
43
44 .global _start
45
46 .text
47 .equ MMIO_SWITCHES, 0x11000000
48 .equ MMIO_SEVSEG, 0x11000040
49 .equ CONST, 32768
50
51 _start:
52 # ~~~~ init reg ~~~~
53 li t1, MMIO_SWITCHES    # t1 is the addr for MMIO SWITCHES
54 li t2, CONST            # t2 is the constant 32768
55 li t3, MMIO_SEVSEG      # t3 is the addr for MMIO 7SEG
56                          # t4 is a test case variable
57
58 # ~~~~ TC injection ~~~~
59 lw t4, TC
60 sw t4, 0(t1)
61
62 # ~~~~ read SWITCHES ~~~~
63 lw t0, 0(t1)            # load SW value into t0
64
65 # ~~~~ unsigned branch ~~~~
66 bgeu t0, t2, Div4
67
68 # ~~ else (false) case ~~
69 slli t0, t0, 1          # multiply by 2
70 j End
71
72 # ~~ if (true) case ~~
73 Div4:
74 srli t0, t0, 2          # logical shift for unsigned
75
76 # ~~~~ write to 7SEG ~~~~
77 End:
78 sw t0, 0(t3)
79
80 # ~~~~ syscall exit status ~~~~
81 li a7, 10               # load syscall num for exit()
82 li a0, 0                # EXIT_SUCCESS code
83 ecall                  # execute sys call
84
85

```

Figure 45: Assembly code for SW2 part 1

Part 2

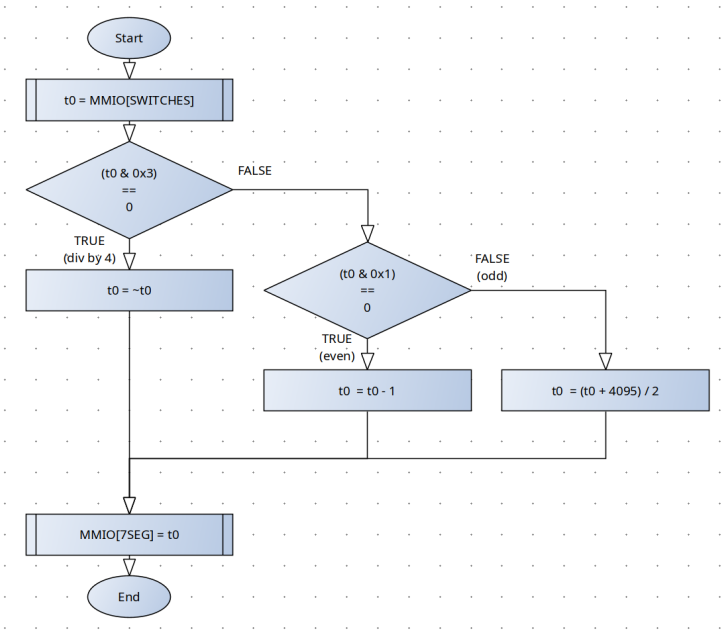


Figure 46: Flow chart for part 2

| TC# | Desc                    | Given MMIO[SWITCHES] | Correct MMIO[7SEG] | Result |
|-----|-------------------------|----------------------|--------------------|--------|
| 1   | Zero is div by 4        | 0                    | 0xFFFFFFFF         | Passed |
| 2   | Two is even             | 0x2                  | 0x1                | Passed |
| 3   | One is odd              | 0x1                  | 0x0800             | Passed |
| 4   | 32-bit unsinged ceiling | 0xFFFFFFFF           | 0x07FF             | Passed |
| 5   | Div by 4 near ceiling   | 0xFFFFFFFFC          | 0x3                | Passed |
| 6   | Odd near ceiling        | 0xFFFFFFFFD          | 0x07FE             | Passed |
| 7   | Even near ceiling       | 0xFFFFFFFEE          | 0xFFFFFFFDD        | Passed |

Table 4: Test cases for part 1

Listing 15: sw2\_p2.asm

```

1  # Author : WhisperingRock
2  #
3  # Purpose : Conditionals, branching, jumping
4  # Input :
5  #     SWITCHES MMIO address to 32-bit unsigned value
6  #     7SEG MMIO address to 8-digits, each of 4-bits
7  #     constant CONST
8  #
9  # Output :
10 #     Write to 7SEG addr
11
12
13 .data
14 # ~~~~ Test Cases ~~~~
15
16 # ~~ TC1 : Zero (Div4) ~~
17 #TC:      .word 0
18 # answer = 0xFFFF_FFFF (verified)
19
20 # ~~ TC2 : Two (Div2 = even) ~~
21 #TC:      .word 2
22 # answer = 0x1 (verified)
23
24 # ~~ TC3 : 1 (odd) ~~
25 #TC:      .word 0x1
26 # answer = 0x0800 (verified)
27
28 # ~~ TC4 : 32-bit unsigned ceiling ~~
29 #TC:      .word 0xFFFFFFFF
30 # answer = 0x0000_07FF | 2047 (verified)
31
32 # ~~ TC5 : unsigned ceiling Div4 ~~
33 #TC:      .word 0xFFFF_FFC
34 # answer = 0x03 (verified)
35
36 # ~~ TC6 : unsigned ceiling odd ~~
37 #TC:      .word 0xFFFF_FFD
38 # answer = 0x07FE (verified)
39
40 # ~~ TC6 : unsigned ceiling even ~~
41 #TC:      .word 0xFFFF_FFE
42 # answer = 0xFFFF_FFD (verified)
43
44 .global _start
45
46 .text
47 .equ MMIO_SWITCHES, 0x11000000
48 .equ MMIO_SEVSEG, 0x11000040
49
50
51 _start:
52 # ~~~~ init reg ~~~~
53 li s1, MMIO_SWITCHES # s1 is the addr for MMIO SWITCHES
54 li s2, MMIO_SEVSEG   # s2 is the addr for MMIO 7SEG
55                       # t0 is the working variable for SWITCHES value
56                       # t1 is a logic case variable
57 li t2, 4095          # t2 is reserved for const
58
59 # ~~~~ TC injection ~~~~
60 lw t0, TC
61 sw t0, 0(s1)
62
63 # ~~~~ read SWITCHES ~~~~
64 lw t0, 0(s1) # load SW value into t0
65
66 # ~~~~ branch : div by 4 ~~~~
67 andi t1, t0, 0x3
68 beqz t1, Div4
69
70 # ~~~~ branch : div by 2 ~~~~
71 andi t1, t0, 0x1
72 beqz t1, Div2
73
74 # ~~ else case ~~
75 add t0, t0, t2 # add const 4095
76 srli t0, t0, 1 # then scale
77 j End
78
79 Div4:
80 not t0, t0 # invert bits
81 j End
82
83 Div2:
84 addi t0, t0, -1
85
86 End: # ~~~~ write to 7SEG ~~~~
87 sw t0, 0(s2)
88
89
90
91 # ~~~~ syscall exit status ~~~~
92 li a7, 10 # load syscall num for exit()37
93 li a0, 0 # EXIT_SUCCESS code
94 ecalls # execute sys call
95
96

```

0.2.3 SW3 - Loops

Assignment:

First, create a flowchart that will perform the following behavior. Then implement the flowchart with OTTER MCU assembly language using only the instructions in the OTTER assembly manual. Make sure your code is formatted properly including comments. Use the RARS simulator to ensure the program performs as desired.

- 1. Read a 32-bit value from SWITCHES (address 0x11000000). Assume the 32-bit value is the combination of two 16-bit unsigned values. Split the 32-bit input into two 16-bit values (the upper and lower 16-bits). Assume both of the 16-bit values are unsigned. Multiply the two 16-bit values together and output the result as a 32-bit unsigned value to 7 SEGMENT (address 0x11000040).
- 2. Read a 32-bit value from SWITCHES (address 0x11000000), delay for 0.5 seconds, and then output the value to 7 SEGMENT (address 0x11000040). The OTTER MCU operates at 25 MHz so each instruction takes 40 ns to run. (Hint: Use loops.) **There is no method to verify the run time in RARS and RARS does not execute at the same speed as the OTTER.** You will need to verify your timing by showing your calculation for the number of instructions being performed between the load (reading from the switches) and store (writing to the 7 segment display) instructions.

Assume the BASYS3 SWITCHES are expanded to 32 individual switches and 7 SEGMENT is expanded to 8 digits.

Figure 48: Questions to address

Part 1

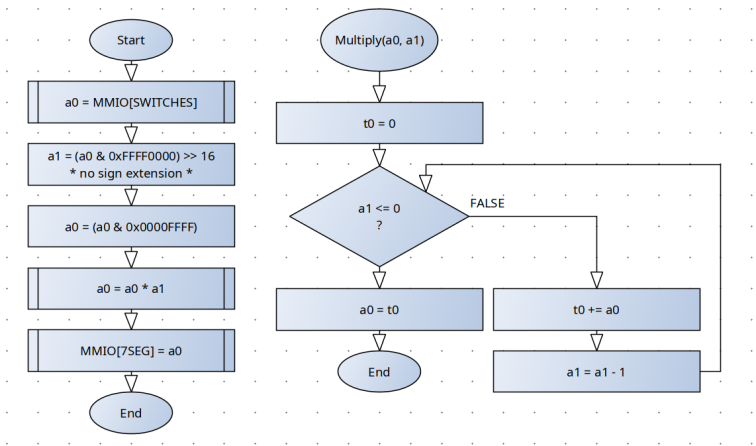


Figure 49: Flow chart for part 1

| TC# | Desc         | Given MMIO[SWITCHES] | Correct MMIO[7SEG] | Result |
|-----|--------------|----------------------|--------------------|--------|
| 1   | Zero         | 0x0000_0000          | 0x0000_0000        | Passed |
| 2   | Zero + One   | 0x0000_0001          | 0x0000_0000        | Passed |
| 3   | One + Zero   | 0x0001_0000          | 0x0000_0000        | Passed |
| 4   | One + One    | 0x0001_0001          | 0x0000_0001        | Passed |
| 5   | num1         | 0x0013_0004          | 0x0000_004C        | Passed |
| 6   | num1 reverse | 0x0004_0013          | 0x0000_004C        | Passed |
| 7   | shift 1      | 0xFFFF_FFFF          | 0xFFFE_0001        | Passed |
| 8   | shift 2      | 0x0010_FFFF          | 0x000F_FFF0        | Passed |
| 9   | shift 3      | 0x0100_FFFF          | 0x00FF_FF00        | Passed |

Table 5: Test cases for part 1

Listing 16: sw3\_p1.asm

```
1 # Author : WhisperingRock
2 #
3 # Purpose : Half word multiply
4 # Input :
5 #     SWITCHES MMIO address to 32-bit unsigned value
6 #     7SEG MMIO address to 8-digits, each of 4-bits
7 #
8 # Output :
9 #     Write to 7SEG addr
10
11
12 .data
13 # ~~~~ Test Cases ~~~~
14
15 # ~~ TC1 : Zero ~~
16 #TC:      .word 0
17 # answer = 0      (verified)
```

```

18
19 # ~~ TC2 : Zero + One ~~
20 #TC:      .word 0x00000001
21 # answer = 0      (verified)
22
23 # ~~ TC3 : One + Zero ~~
24 #TC:      .word 0x00010000
25 # answer = 0      (verified)
26
27 # ~~ TC4 : One ~~
28 #TC:      .word 0x00010001
29 # answer = 1      (verified)
30
31 # ~~ TC5 : num1 ~~
32 #TC:      .word 0x00130004
33 # answer = 0x0000_004C (verified)
34
35 # ~~ TC6 : num1 reverse ~~
36 #TC:      .word 0x00040013
37 # answer = 0x0000_004C (verified)
38
39 # ~~ TC7 : shift1 ~~
40 #TC:      .word 0xFFFFFFFF
41 # answer = 0xFFFE_0001 (verified)
42
43 # ~~ TC8 : shift2 ~~
44 #TC:      .word 0x0010FFFF
45 # answer = 0x000F_FFF0 (verified)
46
47 # ~~ TC9 : shift3 ~~
48 #TC:      .word 0x0100FFFF
49 # answer = 0x00FF_FF00 (verified)
50
51 .global _start
52
53 .text
54 #.equ MMIO_SWITCHES, 0x11000000 # moved to Compact(Text @ 0) config
55 #.equ MMIO_SEVSEG, 0x11000040 # sp moved tp 0x00003FFC
56 #.equ MMIO_SWITCHES, 0x00007F00
57 #.equ MMIO_SEVSEG, 0x00007F40
58 #.equ UPPER_WORD, 0xFFFF0000
59 #.equ LOWER_WORD, 0x0000FFFF
60
61 _start:
62 # ~~~~ init reg ~~~~
63 li s1, MMIO_SWITCHES # s1 is the addr for MMIO SWITCHES
64 li s2, MMIO_SEVSEG # s2 is the addr for MMIO 7SEG
65 # a0 is a working variable for input and return
66 # a1 is a working var for upper half of word
67 # a2 is a working var for lower half of word
68 # t0 is single use TC injection var
69
70 # ~~~~ TC injection ~~~~
71 lw t0, TC
72 sw t0, 0(s1)
73
74 # ~~~~ read SWITCHES ~~~~
75 lw a0, 0(s1)
76
77 # ~~~~ parse input val (and load args for func) ~~~~
78 # ~~ a1 = (a0 & 0xFFFF0000) >> 16 ~~
79 li t0, UPPER_WORD
80 and a1, a0, t0
81 srli a1, a1, 16
82 # ~~ a0 = a0 & 0x0000FFFF ~~
83 li t0, LOWER_WORD
84 and a0, a0, t0
85
86 # ~~~~ caller(_start) saves non-preserved ~~~~
87 # "Not needed b/c we dont need t0"
88 jal Multiply
89
90 # ~~~~ write return to 7SEG ~~~~
91 sw a0, 0(s2)
92
93 # ~~~~ syscall exit ~~~~
94 li a7, 10 # load syscall num for exit()
95 li a0, 0 # EXIT_SUCCESS code
96 ecall # execute sys call
97
98
99 # ~~~~~~
100 # Purpose : Multiply two unsigned 32-bit numbers
101 # Input :
102 # a0 : 32-bit unsigned word
103 # a1 : 32-bit unsigned word
104 # Temps :
105 # t0 :
106 # Note :
107 # - consumes a1
108 # Return:

```

```

109 #      a0 : product of a0 * a1
110 Multiply:
111
112 # ~~~~ calle(Multiply) saves preserved ~~~~
113 # ~~ 1. calle(Multiply) allocates space on stack ~~
114 addi sp, sp, -12
115
116 # ~~ 2. calle(Multiply) stores reg values on stack ~~
117 sw ra, 8(sp)
118 sw s1, 4(sp)
119 sw s2, 0(sp)
120
121 # ~~ 3. calle(Multiply) function execution ~~
122 # ~ sum(t0) init ~
123 addi t0, zero, 0
124 # ~ while a1 <= 0
125 While:
126 bleu a1, zero, EndWhile
127 add t0, t0, a0
128 addi a1, a1, -1
129 j While
130 EndWhile:
131 addi a0, t0, 0
132
133 # ~~ 4. calle(Multiply) restores preserved from stack ~~
134 lw ra, 8(sp)
135 lw s1, 4(sp)
136 lw s2, 0(sp)
137
138 # ~~ 5. calle(Multiply) deallocates space on stack ~~
139 addi sp, sp, 12
140
141 # ~~ calle(Multiply) return ~~
142 jr ra
143 # ~~~~~~

```

## Part 2

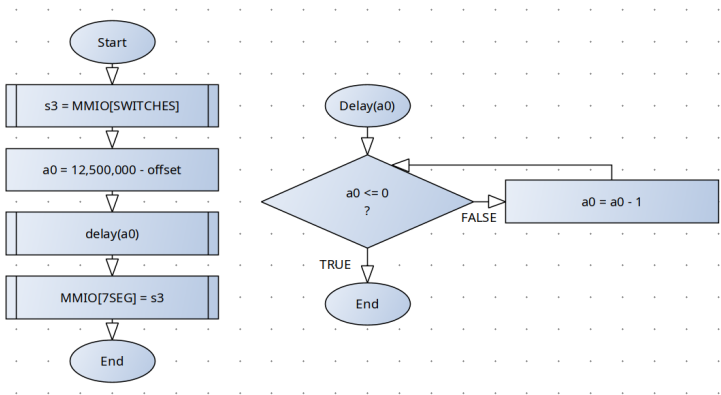


Figure 50: Flow chart for part 2

Consider the following calculations

$$\begin{aligned}
 12.5 \cdot 10^6 \text{ instr} &= 0.5s \cdot \frac{\text{instr}}{40ns} \cdot \frac{10^9ns}{1s} \\
 &= 0x00BE\_BC20
 \end{aligned}$$

- There are 13 instructions, after reading from SWITCHES, that are only executed once. Lets refer to these instructions as bias s.t.  $b = 13$ .
- There are 3 instruction in our delay loop s.t.  $m = 3$ .

Now lets relate our instructions to the instruction layout s.t. we identify how many times we actually need to loop.

$$\begin{aligned}
 3m + b &= 12.5 \cdot 10^9 \\
 m &= \frac{12,500,000 - 13}{3} \\
 &= 4,166,663
 \end{aligned}$$

Rather than use this number in directly, I opted to write the constant in terms of total instructions needed against how many loops we need.

$$\begin{aligned}
\text{offset} &= 12.5 \cdot 10^9 - m \\
&= 8,333,337 \\
&= 0x007F\_2819
\end{aligned}$$

Listing 17: Assembly code for sw3\_p2.asm

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Delay
4 # Input :
5 #     SWITCHES MMIO address to 32-bit unsigned value
6 #     7SEG MMIO address to 8-digits, each of 4-bits
7 #
8 # Output :
9 #     Write to 7SEG addr
10
11
12 .data
13 # ~~~~ Test Cases ~~~~
14
15 # ~~ TC1 : simple ~~
16 TC: .word 0x12345678
17 # answer = 0 (verified)
18
19 .global _start
20
21 .text
22 #.equ MMIO_SWITCHES, 0x11000000 # moved to Compact(Text @ 0) config
23 #.equ MMIO_SEVSEG, 0x11000040 # sp moved tp 0x00003FFC
24 .equ MMIO_SWITCHES, 0x00007F00
25 .equ MMIO_SEVSEG, 0x00007F40
26 .equ HALF_SEC_DELAY, 0x00BEBC20 # 12,500,000 instr for 0.5s delay
27 .equ OFFSET, 0x007F2819 # loop and bias compensation (see notes)
28
29 _start:
30 # ~~~~ init reg ~~~~
31 li s1, MMIO_SWITCHES # s1 is the addr for MMIO SWITCHES
32 li s2, MMIO_SEVSEG # s2 is the addr for MMIO 7SEG
33 # s3 is a working variable for input and return
34 li a0, HALF_SEC_DELAY # a0 is reserved for const
35 li a1, OFFSET # a1 is reserved for offset
36 # t0 is single use TC injection var
37
38 # ~~~~ TC injection ~~~~
39 lw t0, TC
40 sw t0, 0(s1)
41
42 # ~~~~ read SWITCHES ~~~~
43 lw s3, 0(s1)
44
45 # ~~~~ caller(_start) saves non-preserved ~~~~
46 # "Not needed b/c we dont need t0"
47 sub a0, a0, a1 # a0 = delay - offset ; +1 bias
48 jal Delay # +1 bias
49
50 # ~~~~ write return to 7SEG ~~~~
51 sw s3, 0(s2)
52
53 # ~~~~ syscall exit ~~~~
54 li a7, 10 # load syscall num for exit()
55 li a0, 0 # EXIT_SUCCESS code
56 ecall # execute sys call
57
58
59 # ~~~~~~
60 # Purpose : delay(busy wait) of 0.5s
61 # Input :
62 #     a0 : [unsigned 32-bit] delay amount in instruction count
63 # Note :
64 #     - Consuming a0 in the process
65 # Return:
66 #
67
68 Delay:
69
70 # ~~~~ calle(Delay) saves preserved ~~~~
71 # ~~ 1. calle(Delay) allocates space on stack ~~
72 addi sp, sp, -16 # +1 bias
73
74 # ~~ 2. calle(Delay) stores reg values on stack ~~
75 sw ra, 12(sp) # +1 bias
76 sw s1, 8(sp) # +1 bias
77 sw s2, 4(sp) # +1 bias
78 sw s3, 0(sp) # +1 bias
79
80

```

```

81
82 # ~~ 3. calle(Delay) function execution ~~
83 # ~ while a0 <= 0
84 While: # loop = 3x instr
85     bleu a0, zero, EndWhile
86     addi a0, a0, -1
87     j While
88 EndWhile:
89
90 # ~~ 4. calle(Delay) restores preserved from stack ~~
91 lw ra, 12(sp)           # +1 bias
92 lw s1, 8(sp)           # +1 bias
93 lw s2, 4(sp)           # +1 bias
94 lw s3, 0(sp)           # +1 bias
95
96 # ~~ 5. calle(Delay) deallocates space on stack ~~
97 addi sp, sp, 16        # +1 bias
98
99 # ~~ calle(Delay) return ~~
100 jr ra                  # +1 bias
101 # ~~~~~~

```

0.2.4 SW4 - Arrays

**Assignment:**  
First, create a flowchart that will perform the following behavior. Then implement the flowchart with OTTER MCU assembly language using only the instructions in the OTTER assembly manual. Use register alternative names (ie s0, t0) instead of register numbers (ie x5, x10). Make sure your code is formatted properly including comments. Use the RARS simulator to ensure the program performs as desired.

- 1. Using `.data`, define an array in the data segment that contains the first 25 values of the Fibonacci sequence (starting with 0 and ending with 46368). Write a program that progresses through the array and calculates the difference between values that are 3 spots away from each other. For example, the first value (0) and the 4th value (2) is a difference of 2. The next calculation would be between the 2nd (1) and 5th (3) values. When no item exists 3 spots away, no difference can be calculated. Each difference should be output to LEDS (address 0x11000020).
- 2. Create an array of 10 32-bit values. Read 10 32-bit values consecutively from SWITCHES (address 0x11000000) saving them in the array. After the 10th value has been saved, the array should be sorted from least to greatest. The sorted array should be output to LEDS (address 0x11000020) in order from least to greatest.

Assume the BASYS3 SWITCHES and LEDS are expanded to 32 individual switches and LEDs.

Figure 51: Questions to address

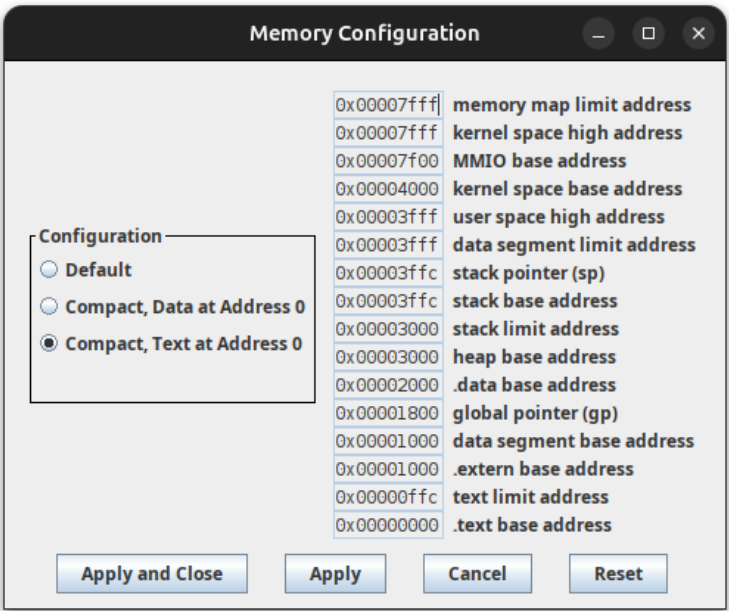


Figure 52: Memory Map for this Assignment

Part 1

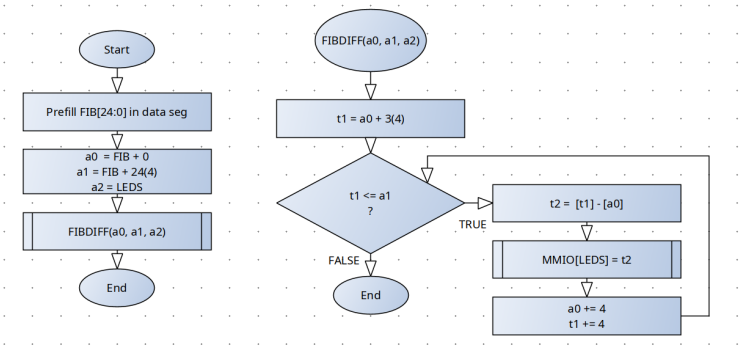


Figure 53: Flow chart for part 1

Listing 18: sw4\_p1.asm

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Difference between array elems
4 # Input :
5 #     Hardcoded values of fibonacci in array
6 #     LEDS MMIO address to 32 bits
7 #
8 # Output :
```

```

9  #       Write to LEDS addr
10
11
12 .data
13 FIB:  .word 0, 1, 1, 2, 3
14       .word 5, 8, 13, 21, 34
15       .word 55, 89, 144, 233, 377
16       .word 610, 987, 1597, 2584, 4181
17       .word 6765, 10946, 17711, 28657, 46368
18
19 # answer = 2, 2, 4, 6, 10, 16, 26, 42, 68, 110, 178,
20 #       288, 466, 754, 1220, 1974, 3194, 5168, 8362
21 #       13530, 21892, 35422
22 # Verified - values match and does not exceed bounds
23
24 .global _start
25
26 .text
27 #.equ MMIO_SWITCHES, 0x00007F00
28 #.equ MMIO_SEVSEG, 0x00007F40
29 #.equ MMIO_LEDS, 0x00007F20
30 #.equ LAST_ELEM, 96 # 25th elem, 4 bytes apart
31
32 _start:
33 # ~~~~ init reg ~~~~
34 la a0, FIB # a0 is the addr for head of FIB
35 addi a1, a0, LAST_ELEM # a1 is the addr for end of FIB
36 li a2, MMIO_LEDS # a2 is the addr for MMIO LEDS
37
38 # ~~~~ caller(_start) saves non-preserved ~~~~
39 # ~~ 1. caller(_start) allocates space on stack ~~
40 addi sp, sp, -24
41 # ~~ 2. caller(_start) stores reg values on stack ~~
42 sw t2, 20(sp)
43 sw t1, 16(sp)
44 sw t0, 12(sp)
45 sw a2, 8(sp)
46 sw a1, 4(sp)
47 sw a0, 0(sp)
48 # ~~ 3. caller(_start) function execution ~~
49 jal FIBDIFF
50 # ~~ 4. caller(_start) restores preserved from stack ~~
51 lw t2, 20(sp)
52 lw t1, 16(sp)
53 lw t0, 12(sp)
54 lw a2, 8(sp)
55 lw a1, 4(sp)
56 lw a0, 0(sp)
57 # ~~ 5. caller(_start) deallocates space on stack ~~
58 addi sp, sp, 24
59
60 # ~~~~ syscall exit ~~~~
61 li a7, 10 # load syscall num for exit()
62 li a0, 0 # EXIT_SUCCESS code
63 ecall # execute sys call
64
65
66 # ~~~~~~
67 # Purpose : For all elements of the array, write (TO LEDS) the difference
68 # between a elem and its left (3 over) neighbor, if they exist.
69 #
70 # Input :
71 # a0 : [unsigned 32-bit] head address of array
72 # a1 : [unsigned 32-bit] tail address of array
73 # a2 : [unsigned 32-bit] LEDS MMIO addr
74 #
75 # Return:
76 #
77
78 FIBDIFF:
79
80 # ~~~~ calle(FIBDIFF) saves preserved ~~~~
81 # ~~ 1. calle(FIBDIFF) allocates space on stack ~~
82 addi sp, sp, -4
83
84 # ~~ 2. calle(FIBDIFF) stores reg values on stack ~~
85 sw ra, 0(sp)
86
87 # ~~ 3. calle(FIBDIFF) function execution ~~
88
89 # ~ init vars ~
90 addi t1, a0, 12 # t1 is the reference index and right neighbor
91 addi t2, zero, 0 # t2 is the window difference
92 addi t3, zero, 0 # t3 is a working variable
93
94 # ~ while the sliding window hasn't hit the right limit ~
95 While: # while t1 <= a1
96 bgtu t1, a1, EndWhile
97
98 # t2 = *(t1) - *(t0)
99 lw t2, 0(t1) # t2 = *(t1) (right neighbor)

```

```

100 lw t3, 0(a0) # t3 = *(a0) (left neighbor)
101 sub t2, t2, t3 # t2 = t2 - t3
102
103 # MMIO[LEDS] = t2
104 sw t2, 0(a2)
105
106 # slide window right
107 addi a0, a0, 4
108 addi t1, t1, 4
109
110 j While
111 EndWhile:
112
113 # ~~ 4. calle(FIBDIFF) restores preserved from stack ~~
114 lw ra, 0(sp)
115
116 # ~~ 5. calle(FIBDIFF) deallocates space on stack ~~
117 addi sp, sp, 4
118
119 # ~~ calle(FIBDIFF) return ~~
120 jr ra
121 # ~~~~~

```

## Part 2

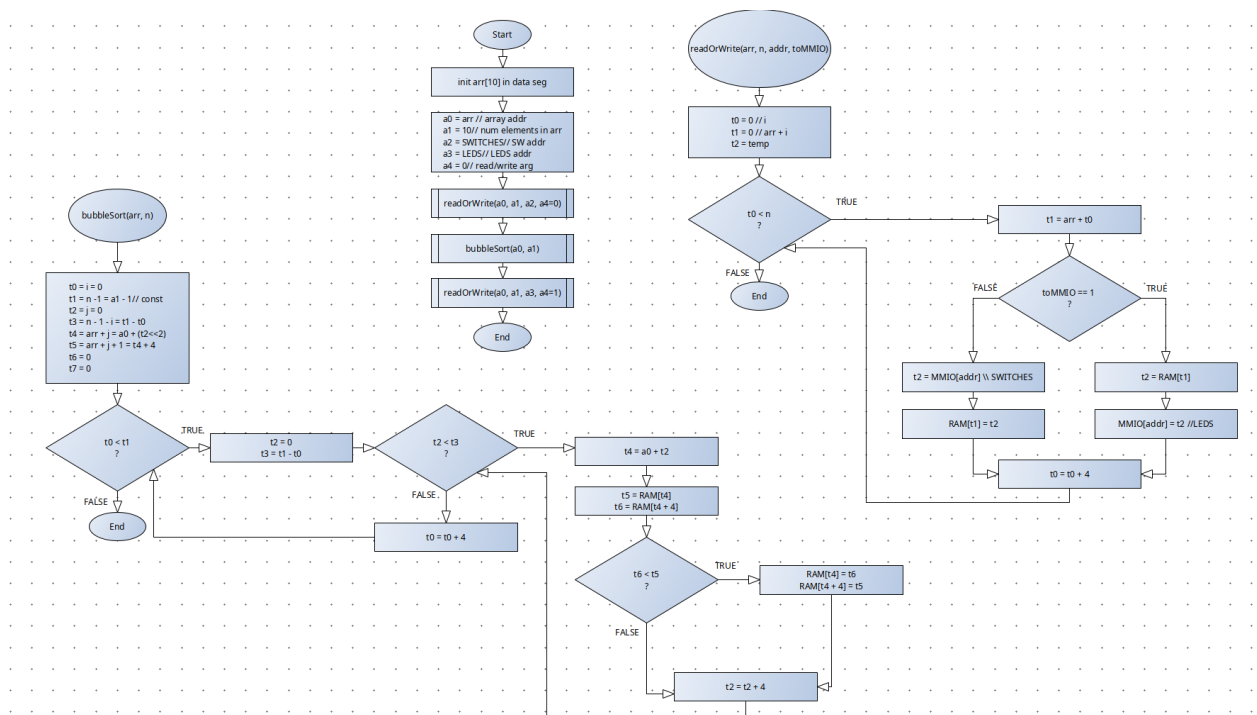


Figure 54: Flow chart for part 2

Listing 19: Assembly code for sw4\_p2.asm

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Read 10 conseq inputs from SWITCHES into array,
4 #           sort array in ASC order,
5 #           then write the the entire array to LEDS consecutively
6 # Input :
7 #         SWITCHES MMIO address to 32 bits
8 #         LEDS MMIO address to 32 bits
9 #
10 # Output :
11 #         Write 10 to LEDS addr
12
13
14 .data
15 ARR: .word -7:10
16
17 # ~~~~ Test Cases ~~~~
18 # ~~ TC1: worst bubble case ~~
19 #TC: .word 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
20 # answer = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
21 # verified r/w and sort
22
23 # ~~ TC2: negatives ~~
24 #TC: .word 1, -5, 2, 8, 3, 3, -1000, 1000, 0, 10
25 # answer = -1000, -5, 0, 1, 2, 3, 3, 8, 10, 1000
26 # verified r/w and sort
27
28 # ~~ TC3: mirror ~~
29 TC: .word 0x7FFFFFFF:4
30 .word 0
31 .word 0xEFFFFFFF:5

```

```

32 # answer = 0xEFFFFFFF:5 , 0x7FFFFFFF:4
33 # verified r/w and sort
34
35 .global _start
36
37 .text
38 .equ MMIO_SWITCHES, 0x00007F00
39 #.equ MMIO_SEVSEG, 0x00007F40
40 .equ MMIO_LEDS, 0x00007F20
41 .equ ARR_N_ELEM, 10
42
43 _start:
44 # ~~~~ init reg ~~~~
45 la s0, ARR # s0 is the addr for head of ARR
46 addi s1, zero, ARR_N_ELEM # s1 is the const num elem in ARR
47 li s2, MMIO_SWITCHES # s2 is the addr for MMIO SWITCHES
48 li s3, MMIO_LEDS # s3 is the addr for MMIO LEDES
49
50
51 # ~~~~ caller(_start) saves non-preserved ~~~~
52 # ~~ 1. caller(_start) allocates space on stack ~~
53 # ~~ 2. caller(_start) stores reg values on stack ~~
54 # ~~ 3. caller(_start) function execution ~~
55 # ~ read from MMIO[SWITCHES] ~~
56 addi a0, s0, 0 # a0 is the base addr for ARR
57 addi a1, s1, 0 # a1 is the num of elem in ARR
58 addi a2, s2, 0 # a2 is the r/w base address (set to SW to read)
59 addi a3, zero, 0 # a3 is the "toMMIO" set to 0 for read
60 jal ReadOrWrite
61 # ~~ 4. caller(_start) restores preserved from stack ~~
62 # ~~ 5. caller(_start) deallocates space on stack ~~
63
64 # ~~~~ caller(_start) saves non-preserved ~~~~
65 # ~~ 1. caller(_start) allocates space on stack ~~
66 # ~~ 2. caller(_start) stores reg values on stack ~~
67 # ~~ 3. caller(_start) function execution ~~
68 # ~ sort array ~~
69 addi a0, s0, 0 # a0 is the base addr for ARR
70 addi a1, s1, 0 # a1 is the num of elem in ARR
71 jal BubbleSort
72 # ~~ 4. caller(_start) restores preserved from stack ~~
73 # ~~ 5. caller(_start) deallocates space on stack ~~
74
75 # ~~~~ caller(_start) saves non-preserved ~~~~
76 # ~~ 1. caller(_start) allocates space on stack ~~
77 # ~~ 2. caller(_start) stores reg values on stack ~~
78 # ~~ 3. caller(_start) function execution ~~
79 # ~ write to MMIO[LEDES] ~~
80 addi a0, s0, 0 # a0 is the base addr for ARR
81 addi a1, s1, 0 # a1 is the num of elem in ARR
82 addi a2, s3, 0 # a2 is the r/w base address (set to LED to write)
83 addi a3, zero, 1 # a3 is the "toMMIO" set to 1 for write
84 jal ReadOrWrite
85 # ~~ 4. caller(_start) restores preserved from stack ~~
86 # ~~ 5. caller(_start) deallocates space on stack ~~
87
88
89 # ~~~~ syscall exit ~~~~
90 li a7, 10 # load syscall num for exit()
91 li a0, 0 # EXIT_SUCCESS code
92 ecall # execute sys call
93
94
95 # ~~~~~~ReadOrWrite()~~~~~
96 # Purpose : Read from from the MMIO address n times and place into array
97 # or write the contents of array onto MMIO addr conseq n times.
98 #
99 #
100 # Input :
101 # a0 : [unsigned 32-bit] base address of array
102 # a1 : [unsigned 32-bit] num elem in array
103 # a2 : [unsigned 32-bit] base addres of MMIO
104 # a3 : [unsigned 32-bit] r/w command (0-read, 1-write)
105 # t3 : [unsigned 32-bit] base addr for test case array (if needed)
106 #
107 # Note :
108 # - modifying a1 to save a register (caller preserve)
109 # Return:
110 #
111
112 ReadOrWrite:
113
114 # ~~~~ calle(READORWRITE) saves preserved ~~~~
115 # ~~ 1. calle(READORWRITE) allocates space on stack ~~
116 addi sp, sp, -20
117
118 # ~~ 2. calle(READORWRITE) stores reg values on stack ~~
119 sw s3, 16(sp)
120 sw s2, 12(sp)
121 sw s1, 8(sp)
122 sw s0, 4(sp)

```

```

123     sw ra, 0(sp)
124
125     # ~~ 3. calle(READORWRITE) function execution ~~
126
127     # ~ init vars ~
128     addi t0, zero, 0      # t0 = i = 0
129     addi t1, zero, 0      # t1 = arr + i
130                                # t2 is local temp
131     la t3, TC             # t3 is a test case var (array base addr)
132                                # t4 is a test case var (dummy)
133
134     slli a1, a1, 2        # shift s.t. we dont use reg for i and 4i
135
136     # ~ for(i=0; i < 10; i++) ~
137 For_ROW:
138     bgeu t0, a1, End_For_ROW
139     add t1, a0, t0        # t1 = arr + 4i
140
141
142     beqz a3, ReadMMIO
143     # ~ write to MMIO addr ~
144     lw t2, 0(t1)         # t2 = *(arr + 4i)
145     sw t2, 0(a2)         # MMIO[LEDS]= t2
146     j End_ReadMMIO
147
148
149 ReadMMIO:
150     # ~~~~ TC injection point ~~~~
151     addi t4, t0, 0        # t4 = 4i
152     add t4, t4, t3        # t4 = TC + 4i
153     lw t4, 0(t4)         # t4 = *(TC + 4i)
154     sw t4, 0(a2)         # MMIO[SWITCHES] = t4
155     # ~~~~~~
156     # ~ read from MMIO addr ~
157     lw t2, 0(a2)         # read SW : t2 = MMIO[SWITCHES]
158     sw t2, 0(t1)         # store in arr : *(arr+4i) = t2
159 End_ReadMMIO:
160
161
162     addi t0, t0, 4        # i++
163     j For_ROW
164 End_For_ROW:
165
166     # ~~ 4. calle(READORWRITE) restores preserved from stack ~~
167     lw s3, 16(sp)
168     lw s2, 12(sp)
169     lw s1, 8(sp)
170     lw s0, 4(sp)
171     lw ra, 0(sp)
172
173     # ~~ 5. calle(READORWRITE) deallocates space on stack ~~
174     addi sp, sp, 20
175
176     # ~~ calle(READORWRITE) return ~~
177     jr ra
178 # ~~~~~~
179
180 # ~~~~~~BubbleSort()~~~~~
181 # Purpose : Sort array in ascending order
182 #
183 #
184 # Input :
185 #     a0 : [unsigned 32-bit] base address of array
186 #     a1 : [unsigned 32-bit] num elem in array
187 #
188 # Note :
189 #
190 # Return:
191 #
192
193 BubbleSort:
194
195     # ~~~~ calle(BubbleSort) saves preserved ~~~~
196     # ~~ 1. calle(BubbleSort) allocates space on stack ~~
197     addi sp, sp, -20
198
199     # ~~ 2. calle(BubbleSort) stores reg values on stack ~~
200     sw s3, 16(sp)
201     sw s2, 12(sp)
202     sw s1, 8(sp)
203     sw s0, 4(sp)
204     sw ra, 0(sp)
205
206     # ~~ 3. calle(BubbleSort) function execution ~~
207
208     slli a1, a1, 2        # shift s.t. we dont use reg for i and 4i
209
210     # ~ init vars ~
211     addi t0, zero, 0      # t0 = i = 0 (4i)
212     addi t1, a1, -4       # t1 = n - 1
213                                # t2 = j = 0

```

```

214             # t3 = n - 1 - i
215             # t4 = arr + j
216             # t5, t6 = dummy
217
218 For_i_Bubble: # for(int i = 0; i < n-1; i++)
219     bgeu t0, t1, End_For_i_Bubble
220
221
222     addi t2, zero, 0           # j = 0
223     sub t3, t1, t0            # n - 1 - i
224 For_j_Bubble: # for(int j = 0; j < n-1-i; j++)
225     bgeu t2, t3, End_For_j_Bubble
226
227     add t4, a0, t2            # t4 = arr + j
228     lw t5, 0(t4)              # t5 = *(arr + j)
229     lw t6, 4(t4)              # t6 = *(arr + j + 1)
230
231     # if(t6 < t5)
232     bgt t6, t5, NoSwap
233     sw t6, 0(t4)              # arr[j] = t6
234     sw t5, 4(t4)              # arr[j+1] = t5
235 NoSwap:
236
237     addi t2, t2, 4            # j++
238     j For_j_Bubble
239 End_For_j_Bubble:
240
241     addi t0, t0, 4            # i++
242     j For_i_Bubble
243 End_For_i_Bubble:
244
245 # ~~ 4. calle(BubbleSort) restores preserved from stack ~~
246 lw s3, 16(sp)
247 lw s2, 12(sp)
248 lw s1, 8(sp)
249 lw s0, 4(sp)
250 lw ra, 0(sp)
251
252 # ~~ 5. calle(BubbleSort) deallocates space on stack ~~
253 addi sp, sp, 20
254
255 # ~~ calle(BubbleSort) return ~~
256 jr ra
257 # ~~~~~

```

0.2.5 SW5 - Division and Stacks

Assignment:

First, create a flowchart that will perform the following behavior. Then implement the flowchart with OTTER MCU assembly language using only the instructions in the OTTER assembly manual. Use register alternative names (ie s0, t0) instead of register numbers (ie x5, x10). Make sure your code is formatted properly including comments. Use the RARS simulator to ensure the program performs as desired.

- 1. Read two 16-bit unsigned values from an array in the data segment. Divide the first value by the 2nd and output the quotient (result) to 7 SEGMENT (address 0x11000040) and the remainder to LEDS (address 0x11000020).
- 2. Read 32-bit values repeatedly from SWITCHES (address 0x11000000). Each value should be added to the stack until the value 0xFFFFFFFF is read. When 0xFFFFFFFF is input the program will not add it to the stack but instead will output all of the saved values to LEDS (address 0x11000020) in the reverse order they were input. Specify the limits of your program. (Assume it is possible for the switches to change between each time they are read)

Assume the BASYS3 SWITCHES and LEDS are expanded to 32 individual switches and LEDs.

Figure 55: Questions to address

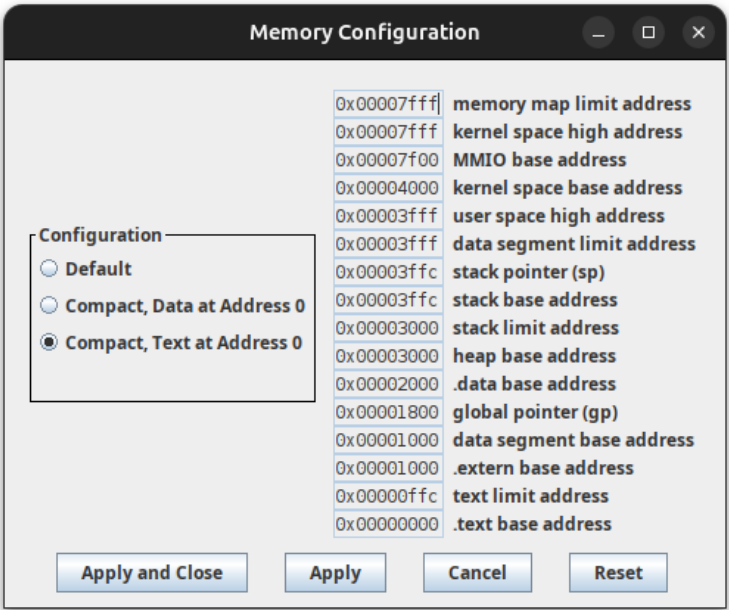


Figure 56: Memory Map for this Assignment

Part 1

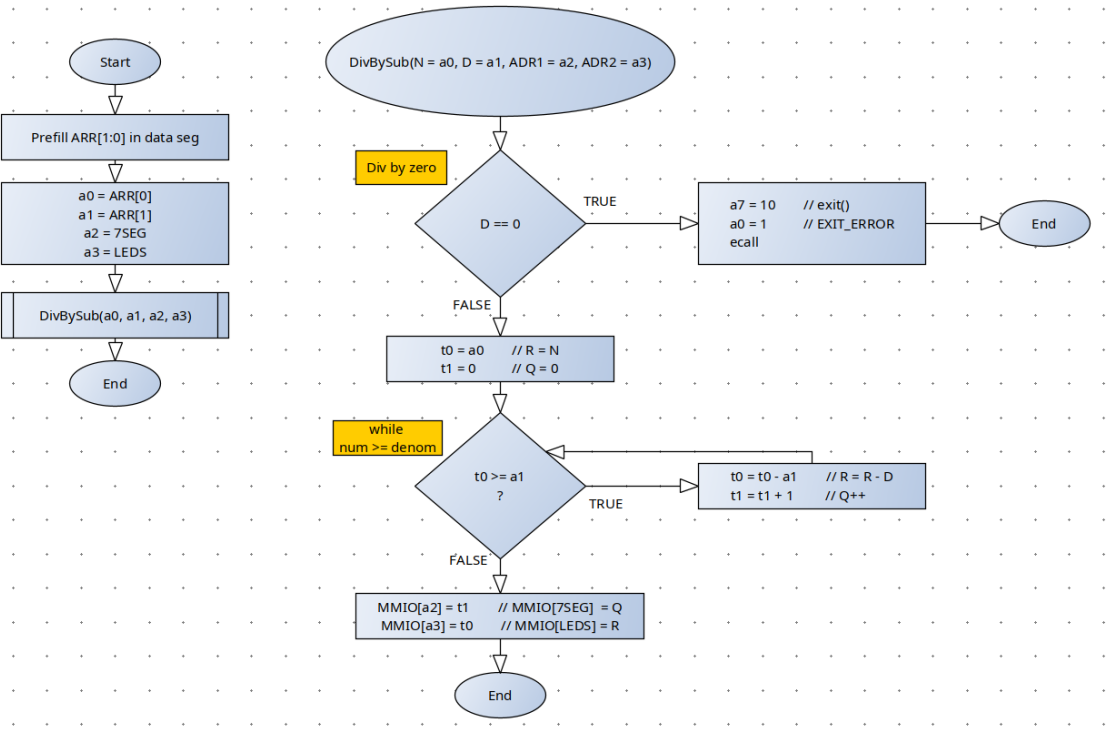


Figure 57: Flow chart for part 1

Listing 20: sw5\_p1.asm

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Division by repeat subtraction
4 # Input :
5 #     - array of two unsigned 16-bit elems
6 #     - 7SEG & LEDS MMIO address to 32 bits
7 #
8 # Output :
9 #     Write to 7SEG and LEDS addr
10
11
12 .data
13 # ~~~~ Test Cases ~~~~
14
15 # ~~ TC1 : 0 / 1 ~~
16 #TC:      .half 0, 1
17 # answer : *verified*
18 # Q = 0
19 # R = 0
20
21 # ~~ TC2 : 1 / 0 ~~
22 #TC:      .half 1, 0
23 # answer : *verified*
24 # RaiseError
25 #
26
27 # ~~ TC3 : 1 / 1 ~~
28 #TC:      .half 1, 1
29 # answer : *verified*
30 # Q = 1
31 # R = 0
32
33 # ~~ TC4 : 2 / 1 ~~
34 #TC:      .half 2, 1
35 # answer : *verified*
36 # Q = 2
37 # R = 0
38
39 # ~~ TC5 : 1 / 2 ~~
40 #TC:      .half 1, 2
41 # answer : *verified*
42 # Q = 0
43 # R = 1
44
45 # ~~ TC6 : half max'd ~~
46 #TC:      .half 0xFFFF, 0xFFFF
47 # answer : *verified*
48 # Q = 1
49 # R = 0
50
51 # ~~ TC7 : quotient max'd ~~
52 #TC:      .half 0xFFFF, 1
53 # answer : *verified*
54 # Q = 0xFFFF
55 # R = 0
56
57 # ~~ TC8 : prime1 ~~
58 #TC:      .half 7919, 13
59 # answer : *verified*
60 # Q = 609
61 # R = 2
62
63 # ~~ TC9 : prime2 ~~
64 #TC:      .half 9973, 241
65 # answer : *verified*
66 # Q = 41
67 # R = 92
68
69 # ~~ TC10 : prime3 ~~
70 #TC:      .half 0xFFFF, 149
71 # answer : *verified*
72 # Q = 439
73 # R = 124
74
75 # ~~ TC11 : 0 / 0 ~~
76 #TC:      .half 0, 0
77 # answer : *verified*
78 # RaiseError
79 #
80
81 # ~~ TC12 : denom max'd ~~
82 #TC:      .half 1, 0xFFFF
83 # answer : *verified*
84 # Q = 0
85 # R = 1
86
87
88 .global _start
89

```

```

90 .text
91 #.equ MMIO_SWITCHES, 0x00007F00
92 #.equ MMIO_SEVSEG, 0x00007F40
93 #.equ MMIO_LEDS, 0x00007F20
94
95 _start:
96 # ~~~~ init reg + TC injection ~~~~
97 la s0, TC # s0 is the addr for ARR
98 li s1, MMIO_SEVSEG # a2 is the addr for MMIO 7SEG
99 li s2, MMIO_LEDS # a2 is the addr for MMIO LEDES
100
101 # ~~~~ caller(_start) saves non-preserved ~~~~
102 # ~~ 1. caller(_start) allocates space on stack ~~
103 # ~~ 2. caller(_start) stores reg values on stack ~~
104 # ~~ 3. caller(_start) function execution ~~
105 lhu a0, 0(s0) # a0 is N
106 lhu a1, 2(s0) # a1 is D; NOTE: halfword spacing
107 addi a2, s1, 0 # a2 is the addr for MMIO 7SEG
108 addi a3, s2, 0 # a3 is the addr for MMIO LEDES
109 jal DivBySub
110 # ~~ 4. caller(_start) restores preserved from stack ~~
111 # ~~ 5. caller(_start) deallocates space on stack ~~
112
113 # ~~~~ syscall exit ~~~~
114 li a7, 10 # load syscall num for exit()
115 li a0, 0 # EXIT_SUCCESS code
116 ecall # execute sys call
117
118
119 # ~~~~~~
120 # Purpose : Perform division by subtraction, writing the quotient to
121 # addr1 and the remainder to addr2
122 #
123 #
124 # Input :
125 # a0 : [unsigned 16-bit] Numerator variable N
126 # a1 : [unsigned 16-bit] Denominator variable D
127 # a2 : [unsigned 32-bit] addr1
128 # a3 : [unsigned 32-bit] addr2
129 #
130 # Return:
131 #
132
133 DivBySub:
134
135 # ~~~~ calle(DivBySub) saves preserved ~~~~
136 # ~~ 1. calle(DivBySub) allocates space on stack ~~
137 addi sp, sp, -16
138
139 # ~~ 2. calle(DivBySub) stores reg values on stack ~~
140 sw s2, 12(sp)
141 sw s1, 8(sp)
142 sw s0, 4(sp)
143 sw ra, 0(sp)
144
145 # ~~ 3. calle(DivBySub) function execution ~~
146
147 # ~ error ~
148 blez a1, RaiseError
149
150 # ~ init ~
151 addi t0, a0, 0 # R = t0 = N
152 addi t1, zero, 0 # Q = t1 = 0
153
154 While: # ~ while div produces integers s.t. (R >= D) ~
155 # (R>= D) = (D > R)
156 bgtu a1, t0, EndWhile
157 sub t0, t0, a1 # R = R - D
158 addi t1, t1, 1 # Q++
159 j While
160 EndWhile:
161 # ~ write to MMIOs ~
162 sw t1, 0(a2) # Q goes to addr1
163 sw t0, 0(a3) # R goes to addr2
164
165 # ~~ 4. calle(DivBySub) restores preserved from stack ~~
166 lw s2, 12(sp)
167 lw s1, 8(sp)
168 lw s0, 4(sp)
169 lw ra, 0(sp)
170
171 # ~~ 5. calle(DivBySub) deallocates space on stack ~~
172 addi sp, sp, 16
173
174 # ~~ calle return ~~
175 jr ra
176 # ~~~~~~
177
178 RaiseError:
179 # ~~~~ syscall exit ~~~~
180 li a7, 10 # load syscall num for exit()

```

```

181  li a0, 1      # EXIT_FAILURE code
182  ecall        # execute sys call

```

Part 2

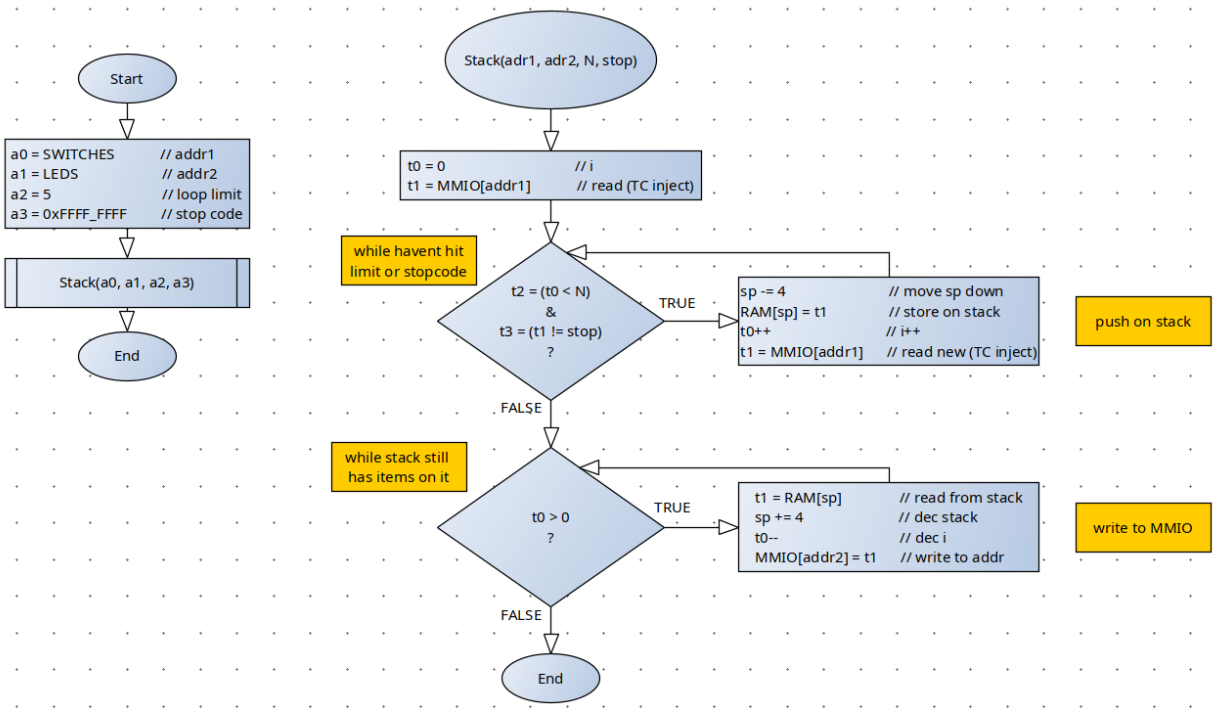


Figure 58: Flow chart for part 2

Listing 21: Assembly code for sw5\_p2.asm

```

1  # Author : WhisperingRock
2  #
3  # Purpose : Stack using the stack
4  # Input :
5  #     - SWITCHES & LEDS MMIO address to 32 bits
6  #
7  # Output :
8  #     - conseq write to LEDS addr
9
10
11 .data
12 # ~~~~ Test Cases ~~~~
13
14 # ~~ TC1 : loop limited 1 ~~
15 #TC:      .word 5, 4, 3, 2, 1
16 # answer : 1, 2, 3, 4, 5 (conseq writes)
17 # verified!
18
19 # ~~ TC2 : stopcode limited 1 ~~
20 #TC:      .word 5, 4, 0xFFFFFFFF, 2, 1
21 # answer : 4, 5 (conseq writes)
22 # verified!
23
24 # ~~ TC3 : immediate stopcode ~~
25 #TC:      .word 0xFFFFFFFF, 5, 4, 3, 2, 1
26 # answer : *no writes*
27 # verified!
28
29 # ~~ TC4 : loop limit is zero ~~
30 # ~ NOTE : change LIMIT to 0 ~
31 #TC:      .word 5, 4, 3, 2, 1
32 # answer : *no writes*
33 # verified!
34
35 # ~~ TC5 : near ceiling ~~
36 #TC:      .word 0xFFFFFFF, 0xFFFFFFF, 0xFFFFFFF, 0xFFFFFFF, 0xFFFFFFF, 0
37 # answer : 0xFFFFFFF, 0xFFFFFFF, 0xFFFFFFF, 0xFFFFFFF, (conseq writes)
38 # verified!
39
40 # ~~ TC6 : stopcode and loop limits at same time ~~
41 #TC:      .word 5, 4, 3, 2, 1, 0xFFFFFFFF, 999
42 # answer : 1, 2, 3, 4, 5 (conseq writes)
43 # verified!
44
45 # ~~ TC7 : beyond loop limit ~~
46 #TC:      .word 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
47 # answer : 6, 7, 8, 9, 10 (conseq writes)
48 # verified!
49
50 .global _start

```

```

51
52 .text
53
54 .equ MMIO_SWITCHES, 0x00007F00
55 #.equ MMIO_SEVSEG, 0x00007F40
56 .equ MMIO_LEDS, 0x00007F20
57
58 .equ LIMIT, 5
59 .equ STOPCODE, 0xFFFFFFFF
60
61 _start:
62 # ~~~~ init reg + TC injection ~~~~
63 la s0, TC # s0 is the addr for TC array
64 li s1, MMIO_SWITCHES # a2 is the addr for MMIO SWITCHES
65 li s2, MMIO_LEDS # a2 is the addr for MMIO LEDs
66
67 # ~~~~ caller(_start) saves non-preserved ~~~~
68 # ~~ 1. caller(_start) allocates space on stack ~~
69 # ~~ 2. caller(_start) stores reg values on stack ~~
70 # ~~ 3. caller(_start) function execution ~~
71 addi a0, s1, 0 # addr1
72 addi a1, s2, 0 # addr2
73 li a2, LIMIT # loop limit
74 li a3, STOPCODE # stop value
75 jal Stack
76 # ~~ 4. caller(_start) restores preserved from stack ~~
77 # ~~ 5. caller(_start) deallocates space on stack ~~
78
79 # ~~~~ syscall exit ~~~~
80 li a7, 10 # load syscall num for exit()
81 li a0, 0 # EXIT_SUCCESS code
82 ecall # execute sys call
83
84
85 # ~~~~~~
86 # Purpose : Read values from addr1 onto the stack until the loop limit
87 # or stop code is reached. Then write the values, from the stack
88 # to addr2.
89 #
90 #
91 # Input :
92 # a0 : [unsigned 32-bit] read address
93 # a1 : [unsigned 32-bit] write address
94 # a2 : [unsigned 32-bit] loop limit
95 # a3 : [unsigned 32-bit] stop code
96 #
97 # Note :
98 # - using s0 as a global for TC injection
99 # Return:
100 #
101
102 Stack:
103
104 # ~~~~ calle(Stack) saves preserved ~~~~
105 # ~~ 1. calle(Stack) allocates space on stack ~~
106 addi sp, sp, -16
107
108 # ~~ 2. calle(Stack) stores reg values on stack ~~
109 sw s2, 12(sp)
110 sw s1, 8(sp)
111 sw s0, 4(sp)
112 sw ra, 0(sp)
113
114 # ~~ 3. calle(Stack) function execution ~~
115 # ~ init ~
116 addi t0, zero, 0 # t0 = i = 0
117 # t1, t2, t3 = read and logic temps
118 # t4 = TC arr index variable
119 # ---- TC Injection ----
120 slli t4, t0, 2 # t4 = 4i (TC)
121 add t4, t4, s0 # t4 = TC + 4i
122 lw t4, 0(t4) # t4 = *(TC + 4i)
123 sw t4, 0(a0) # MMIO[addr1] = t4
124 # -----
125 lw t1, 0(a0) # t1 = MMIO[addr1]
126 ReadWhile:
127 sltu t2, t0, a2 # t2 = (i < limit)
128 seqz t2, t2
129 xor t3, t1, a3 # t3 = (t1 != stopcode)
130 seqz t3, t3
131 or t3, t3, t2 # t3 = t3 && t2
132 bgt t3, zero, End_ReadWhile # while (i < limit) && (t1 != stopcode)
133 addi sp, sp, -4 # sp -= 4
134 sw t1, 0(sp) # RAM[sp] = read
135 addi t0, t0, 1 # i++
136 # ---- TC Injection ----
137 slli t4, t0, 2 # t4 = 4i (TC)
138 add t4, t4, s0 # t4 = TC + 4i
139 lw t4, 0(t4) # t4 = *(TC + 4i)
140 sw t4, 0(a0) # MMIO[addr1] = t4
141 # -----

```

```

142     lw t1, 0(a0)           # t1 = MMIO[addr1]
143     j ReadWhile
144 End_ReadWhile:
145 WriteWhile:
146     beqz t0, End_WriteWhile # while(i > 0)
147     lw t1, 0(sp)           # t1 = RAM[sp]
148     addi sp, sp, 4         # sp += 4
149     addi t0, t0, -1        # i--
150     sw t1, 0(a1)          # MMIO[addr2] = t1
151     j WriteWhile
152 End_WriteWhile:
153
154 # ~~ 4. calle(Stack) restores preserved from stack ~~
155 lw s2, 12(sp)
156 lw s1, 8(sp)
157 lw s0, 4(sp)
158 lw ra, 0(sp)
159
160 # ~~ 5. calle(Stack) deallocates space on stack ~~
161 addi sp, sp, 16
162
163 # ~~ calle return ~~
164 jr ra
165 # ~~~~~
166
167 RaiseError:
168 # ~~~~ syscall exit ~~~~
169 li a7, 10      # load syscall num for exit()
170 li a0, 1       # EXIT_FAILURE code
171 ecall         # execute sys call

```

0.3    Appendix